

UNIT IV:

Deadlocks:

- Definition,
- Necessary Conditions for Deadlock,

Methods For Handling Deadlocks:

- Deadlock Prevention,
- Avoidance(Bankers Algorithm),
- Detection and Deadlock recovery.

Memory Management:

- Logical versus Physical Address Space,
- Swapping,
- Contiguous memory allocation,
- Paging and Segmentation techniques,
- Segmentation with paging,

Virtual Memory: Demand Paging,

Page-Replacement Algorithms: FIFO, LRU and Optimal, Thrashing.

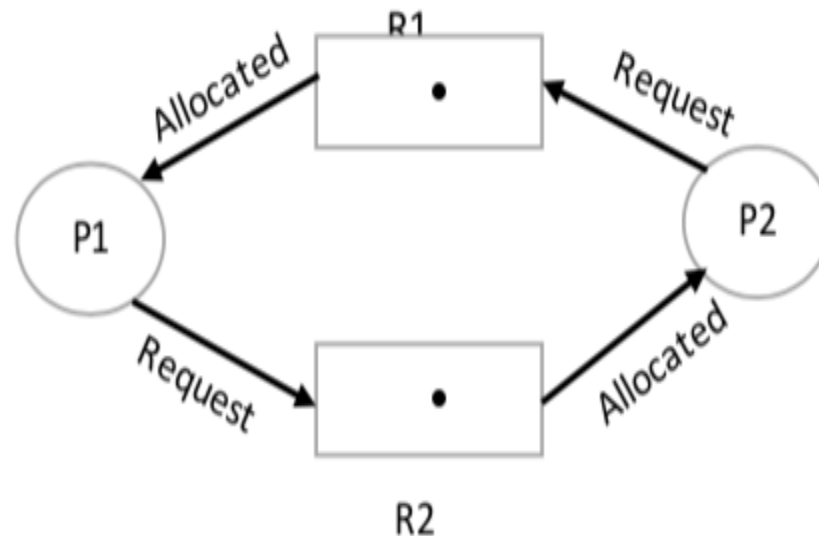
Unit IV

Deadlocks

- Necessary Conditions for Deadlocks
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Deadlock Recovery

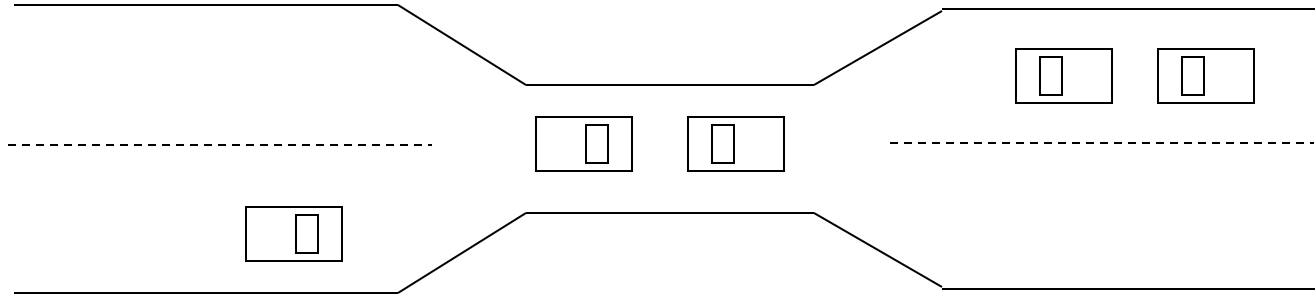
Deadlocks

- In a multiprogramming environment, several processes may compete for a finite number of resources.
- A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a **deadlock**.



- Deadlocks may also involve different resource types. For example, consider a system with one printer and one DVD drive. Suppose that process P_i is holding the DVD and process P_j is holding the printer. If P_i requests the printer and P_j requests the DVD drive, a deadlock occurs.

Bridge Crossing Example



- Traffic only in One Direction.
- Each Section of a Bridge can be viewed as a Resource.
- If a Deadlock occurs, it can be resolved if one car backs up (Preempt Resources and Rollback).
- Several Cars may have to be backed up if a Deadlock Occurs.
- **Starvation is Possible.**

- Under the normal mode of operation, a process may utilize a resource in only the following sequence:

1. Request: The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.

2. Use: The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).

3. Release: The process releases the resource.

- The request and release of resources may be system calls, as explained in unit 1. Examples are the request() and release() device, open() and close() file, and allocate() and free() memory system calls.

➤ **Deadlock Characterization- *Necessary Conditions for Deadlocks***

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
- **2. Hold and wait.** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

- **3. No preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **4. Circular wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for a resource held by P_n , and P_n is waiting for a resource held by P_0 .

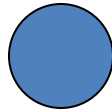
➤ Resource-Allocation Graph

- Deadlocks can be described more precisely in terms of a directed graph called a **system Resource-Allocation Graph(RAG)**.
- **This graph consists of a set of *vertices* V and a set of edges E .**
- The set of vertices V is partitioned into two different types of nodes:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system, and
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.

- A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$;
- it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.
- A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$; it signifies that an instance of resource type R_j has been allocated to process P_i .
- A **directed edge** $P_i \rightarrow R_j$ is called a **request edge**;
- A **directed edge** $R_j \rightarrow P_i$ is called an **assignment edge**.
- Pictorially, we represent each process P_i as a **circle** and each resource type R_j as a **rectangle**.
- Since resource type R_j may have more than one instance, we represent each such instance as a **dot** within the rectangle.

➤ Resource-Allocation Graph (Cont.)

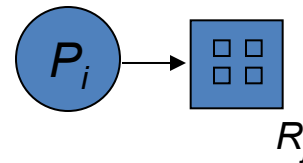
- Process



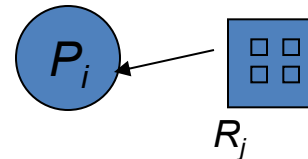
- Resource Type with 4 instances



- P_i requests instance of R_j



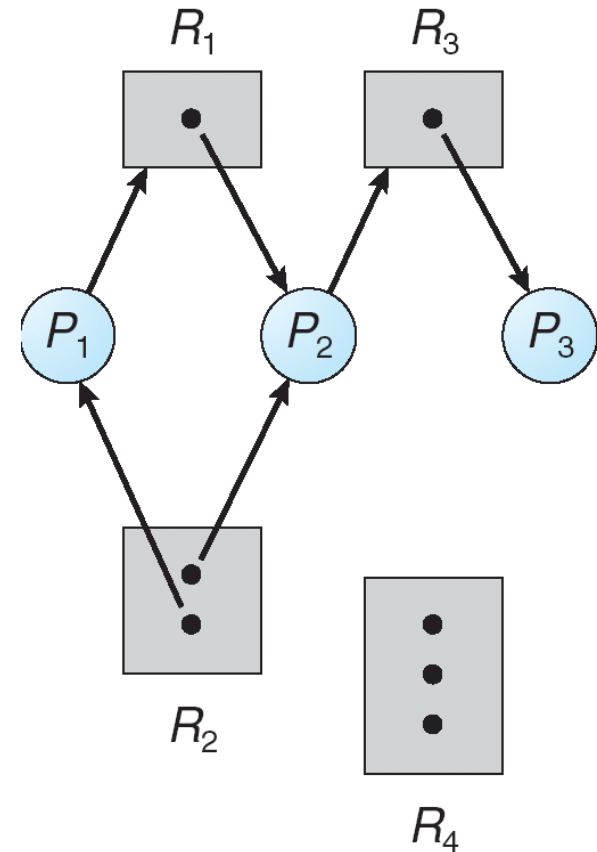
- P_i is holding an instance of R_j



Example of a Resource Allocation Graph

The resource-allocation graph shown in Figure depicts the following situation.

- The sets P , R , and E :
 - $P = \{P1, P2, P3\}$
 - $R = \{R1, R2, R3, R4\}$
 - $E = \{P1 \rightarrow R1, P2 \rightarrow R3, R1 \rightarrow P2, R2 \rightarrow P2, R2 \rightarrow P1, R3 \rightarrow P3\}$



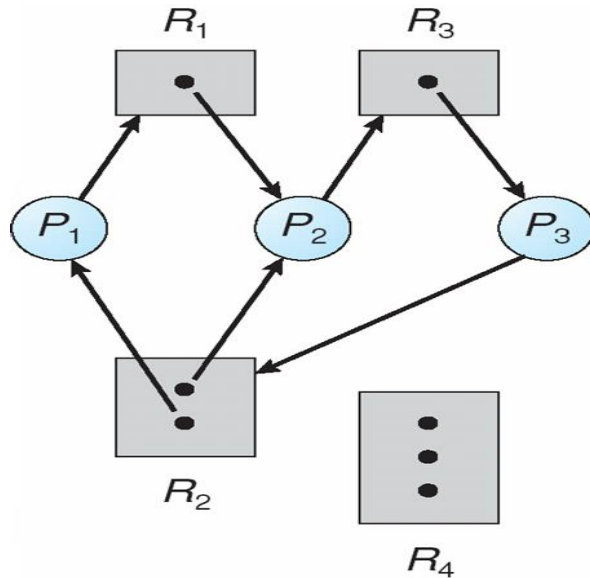
- **Resource instances:**

- One instance of resource type $R1$
- Two instances of resource type $R2$
- One instance of resource type $R3$
- Three instances of resource type $R4$

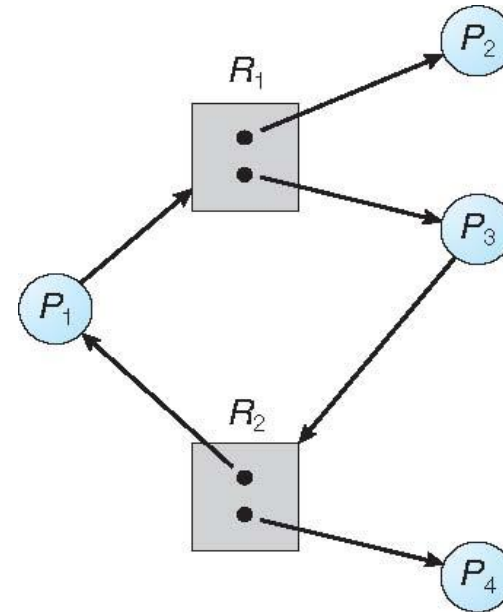
Process states:

- Process $P1$ is holding an instance of resource type $R2$ and is waiting for an instance of resource type $R1$.
- Process $P2$ is holding an instance of $R1$ and an instance of $R2$ and is waiting for an instance of $R3$.
- Process $P3$ is holding an instance of $R3$.

Resource Allocation Graph With A Deadlock



Graph With A Cycle But No Deadlock



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - ✓ if only one instance per resource type, then deadlock
 - ✓ if several instances per resource type, possibility of deadlock

➤ Methods for Handling Deadlocks



- **Deadlock Ignorance (Ostrich method)**
 - **Deadlock Prevention**
 - **Deadlock Avoidance**
 - **Deadlock Detection and recovery**
-
- **Deadlock Ignorance (Ostrich method)**
 - Ensure that the system will *never* enter a deadlock state.
 - Allow the system to enter a deadlock state and then recover.
 - Ignore the problem and pretend that deadlocks never occur in the system;
 - Used by most Operating Systems, including UNIX.

➤ *Deadlock Prevention*



Mutual Exclusion

Not required for sharable resources; must hold for nonsharable resources.

Hold and Wait

- Must guarantee that whenever a process requests a resource, it does not hold any other resources.
- Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none.
- Low resource utilization; starvation possible.

No Preemption

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
- Preempted resources are added to the list of resources for which the process is waiting.
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Circular Wait

Impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

Deadlock Avoidance



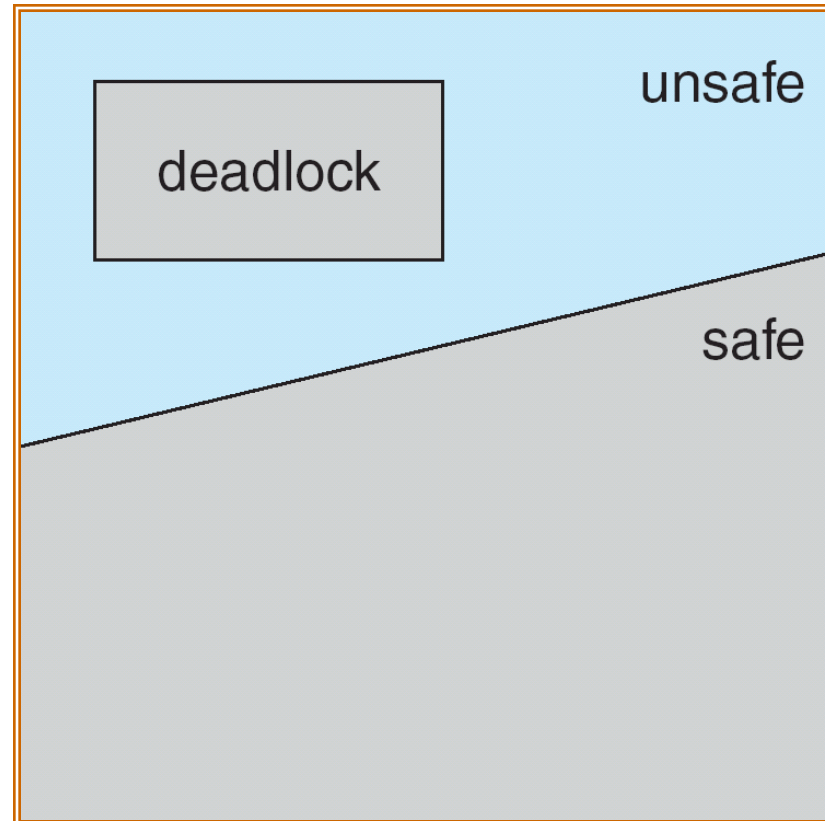
Safe State

Deadlock Avoidance (Banker's) Algorithm

- Example 1
- Example 2
- Example 3

➤ *Basic Facts*

- If a system is in safe state $\Rightarrow$ no deadlocks.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock.
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.



❖ *Deadlock Avoidance*

Requires that the system has some additional *a priori* information available.

- Simplest and most useful model requires that each process declare the *maximum no.* of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation *state* is defined by the no. of available and allocated resources, and the maximum demands of the processes.

Banker's Algorithm

- Multiple instances.
- Each process must make a priori claim of maximum use.
- When a process requests a resource it may have to wait.
- When a process gets all its resources it must return them in a finite amount of time.

❖ *Banker's Algorithm*

- When a new process enters the system, it must declare maximum no. of instances of each resource type, it may need. This no. may not exceed the total no. of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of the resources, leave the system in a safe state.
- If it will, the resources are allocated, otherwise, the process must wait until some other process releases enough resources.

➤ *Data Structures for the Banker's Algorithm*

Let

n = No. of processes

m = No. of resources types.

- **Available:**

- Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available.

- **Max:**

- $n \times m$ matrix.
- If $Max [i,j] = k$, then process P_i may request at most k instances of resource type R_j .

- **Allocation:**

- $n \times m$ matrix.
- If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j .

- **Need:**

- $n \times m$ matrix.
- If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task.

$$Need [i,j] = Max[i,j] - Allocation [i,j]$$

➤ *Safety Algorithm*

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

$$Work = Available$$

$$Finish[i] = false \text{ for } i = 0, 1, \dots, n-1.$$

2. Find an i such that both:

$$(a) \text{ } Finish[i] = false$$

$$(b) \text{ } Need_i \leq Work$$

If no such i exists, go to step 4.

3. $Work = Work + Allocation_i$

$$Finish[i] = true$$

go to step 2.

4. If $Finish[i] == true$ for all i , then the system is in a safe state.

➤ *Resource–Request Algorithm for Process P_i*

$Request_i$ = request vector for process P_i .

⌞⌞

If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$ go to step 2.
Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3.
Otherwise P_i must wait, since resources are not available.
3. Pretend to allocate requested resources to P_i
by modifying the state as follows:
 $Available = Available - Request_i$;
 $Allocation_i = Allocation_i + Request_i$;
 $Need_i = Need_i - Request_i$;

If *safe* $\Rightarrow$ the resources are allocated to P_i . (Banker's Safety Algo for Avoidance)

If *unsafe* $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

➤ *Example of Banker's Algorithm*

1. Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

- What is the content of the matrix *Need*?
- Is the system in a *Safe state*?
- If a request from process P₁ arrive for (1, 0, 2) can the request be granted immediately?

➤ *Example of Banker's Algorithm*

1. Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

- What is the content of the matrix *Need*?
- Is the system in a *Safe state*?
- If a request from process P₁ arrive for (1, 0, 2) can the request be granted immediately?

a. Content of the matrix *Need*

Need = Max – Allocation

	<u>Allocation</u>	<u>Max</u>	<u>Need</u>
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	7 4 3
P ₁	2 0 0	3 2 2	1 2 2
P ₂	3 0 2	9 0 2	6 0 0
P ₃	2 1 1	2 2 2	0 1 1
P ₄	0 0 2	4 3 3	4 3 1

b. Is the system in a safe state?

If ($\text{Need} \leq \text{Available}$)

$\text{Available} = \text{Available} + \text{Allocated}$

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P ₀	0 1 0	7 4 3	3 3 2 7 5 5
P ₁	2 0 0	1 2 2	5 3 2
P ₂	3 0 2	6 0 0	10 5 5
P ₃	2 1 1	0 1 1	7 4 3
P ₄	0 0 2	4 3 1	7 4 5

Sequence is $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ and the System is in *Safe State*.

c. If a request from process P_1 arrive for (1, 0, 2) can the request be granted immediately?

If $(Request_i \leq Need_i)$, if $(Request_i \leq Available)$

⚡

$Available = Available - Request_i$

$Allocation_i = Allocation_i + Request_i$

$Need_i = Need_i - Request_i$ $i = 1, Request_1 = 1\ 0\ 2$

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>	
	A B C	A B C	A B C	
P_0	0 1 0	7 4 3	<u>3 3 2</u> (2 3 0)	7 5 5
P_1	<u>2 0 0</u> (3 0 2)	<u>1 2 2</u> (0 2 0)	5 3 2	
P_2	3 0 2	6 0 2		10 5 5
P_3	2 1 1	0 1 1	7 4 3	
P_4	0 0 2	4 3 1	7 4 5	

Sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ and the system is in **Safe State**.

Hence, We can **immediately grant the request of process P_1** .

Deadlock Detection

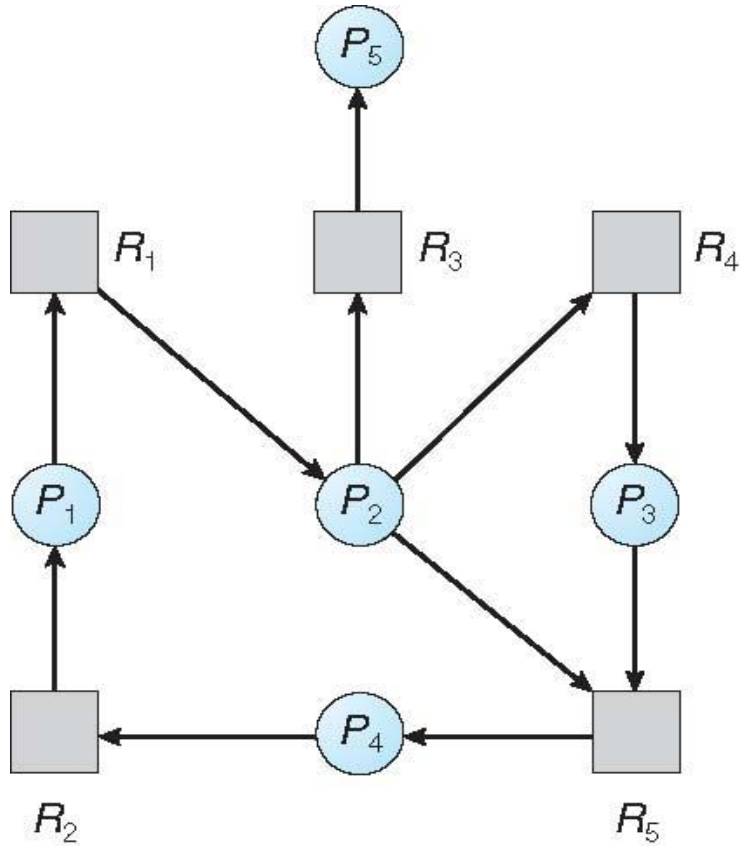


- Allow System to enter Deadlock State
- Detection Algorithm
- Recovery Scheme

Detection- Single Instance of Each Resource Type

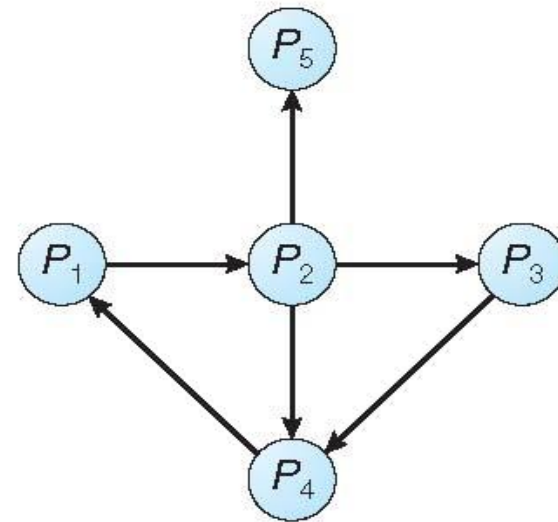
1. Maintain *wait-for* graph
 - Nodes are processes
 - $P_i \rightarrow P_j$ if P_i is waiting for P_j
2. Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
3. An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

Resource-Allocation Graph and Wait-for Graph



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph

➤ Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $Request[i][j] = k$, then process P_i is requesting k more instances of resource type R_j .

Detection Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively Initialize:
 - (a) $Work = Available$
 - (b) For $i = 1, 2, \dots, n$,
if $Allocation_i \neq 0$, then $Finish[i] = false$;
otherwise, $Finish[i] = true$
2. Find an index i such that both:
 - (a) $Finish[i] == false$
 - (b) $Request_i \leq Work$If no such i exists, go to step 4
3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2
4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state

Example of Banker's Algorithm

2. Consider the following snapshot of a system:



	<u>Allocation</u>				<u>Max</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P ₀	0	0	1	2	0	0	1	2	1	5	2	0
P ₁	1	0	0	0	1	7	5	0				
P ₂	1	3	5	4	2	3	5	6				
P ₃	0	6	3	2	0	6	5	2				
P ₄	0	0	1	4	0	6	5	6				

Answer the following questions using the banker's algorithm.

- What is the content of the matrix *Need*?
- Is the system in a *Safe state*?
- If a request from process P₁ arrive for (0, 4, 2, 0) can the request be granted immediately?

System in a Safe State



If ($\text{Need} \leq \text{Available}$)

Available = Available + Allocated

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>	
	<i>a b c d e</i>	<i>a b c d e</i>	<i>a b c d e</i>	
Process A	1 0 2 1 1	0 1 0 0 2	0 0 x 1 1	5 2 6 5 3
Process B	2 0 1 1 0	0 2 1 0 0	4 2 4 4 1	4 2 4 4 2
Process C	1 1 0 1 0	1 0 3 0 0	2 2 3 3 1	
Process D	1 1 1 1 0	0 0 1 1 1	1 1 2 2 1	

Sequence of Processes is **D, C, B, A** and the System is in *Safe State*.

Content of the matrix *Need*

$$\mathbf{Need} = \mathbf{Maximum} - \mathbf{Allocation}$$

	<u>Allocation</u>					<u>Maximum</u>					<u>Need</u>				
	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
Process A	1	0	2	1	1	1	1	2	1	3	0	1	0	0	2
Process B	2	0	1	1	0	2	2	2	1	0	0	2	1	0	0
Process C	1	1	0	1	0	2	1	3	1	0	1	0	3	0	0
Process D	1	1	1	1	0	1	1	2	2	1	0	0	1	1	1

➤ Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i

Example (Cont.)

- P_2 requests an additional instance of type C

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4

➤ Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

Deadlock Recovery



- Process Termination
- Resource Preemption

Deadlock Recovery: Process Termination

- Abort all Deadlocked Processes.
- Abort one process at a time until the Deadlock Cycle is eliminated.

In which order should we choose to abort?

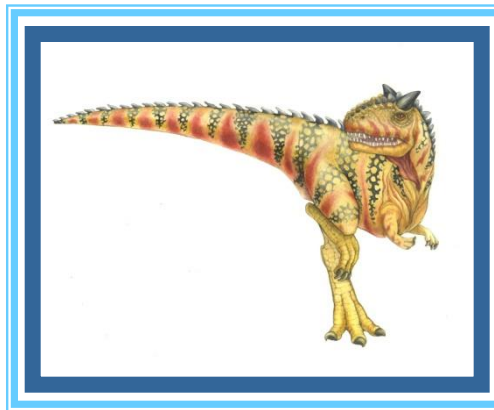
- Priority of the process.
- How long process has computed, and how much longer to completion.
- Resources the process has used.
- Resources process needs to complete.
- How many processes will need to be terminated.
- Is process interactive or batch?

Deadlock Recovery: Resource Preemption

- Selecting a Victim
 - Minimize Cost.
- Rollback
 - Return to some Safe State,
Restart Process for that state.
- Starvation
 - Same Process may always be picked as Victim, include no. of Rollback in Cost factor.



Memory Management

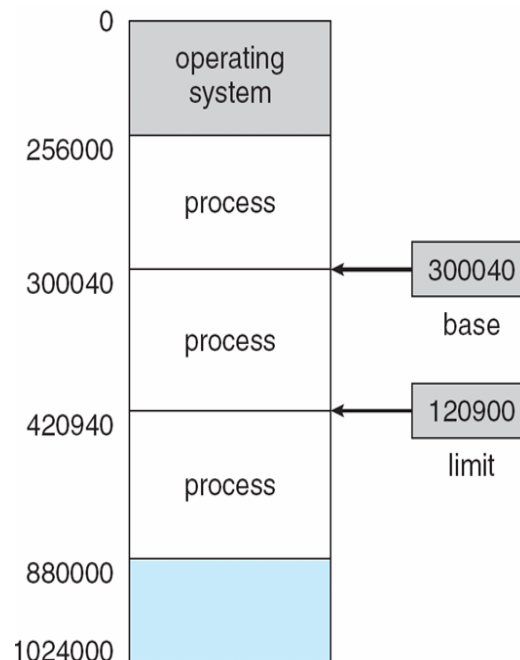


Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Base and Limit Registers

- A pair of **base** and **limit registers** define the logical address space
- The base register holds the smallest legal physical memory address.
- The limit register specifies the size of the range.
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



Types of Addresses used in a program

□ Symbolic addresses

The addresses used in a source code. The variable names, constants, and instruction labels are the basic elements of the symbolic address space. E.g. an integer variable called *count*.

□ Relative addresses

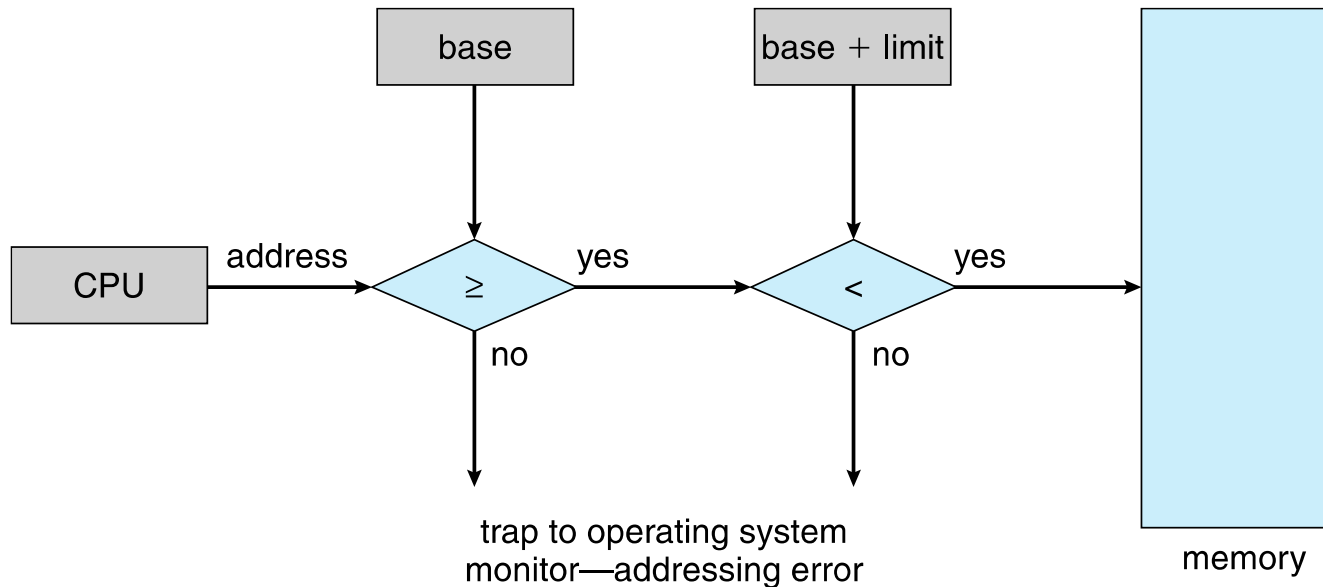
At the time of compilation, a compiler converts symbolic addresses into relative addresses.

□ Physical or absolute addresses

The loader generates these addresses at the time when a program is loaded into main memory.

Hardware Address Protection

- The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction.
- Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.



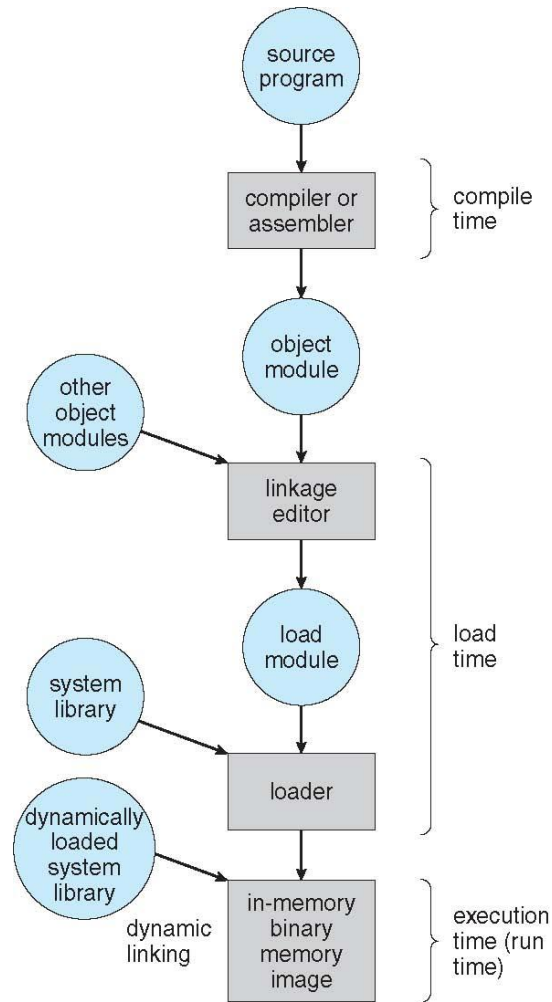
Address Binding

- Programs on disk, ready to be brought into memory to execute form an **input queue**
 - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
 - How can it not be?
- Further, addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic
 - Compiled code addresses **bind** to relocatable addresses
 - i.e. “14 bytes from beginning of this module”
 - Linker or loader will bind relocatable addresses to absolute addresses
 - i.e. 74014
 - Each binding maps one address space to another

Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another
 - Need hardware support for address maps (e.g., base and limit registers)

Multistep Processing of a User Program



Logical vs. Physical Address Space

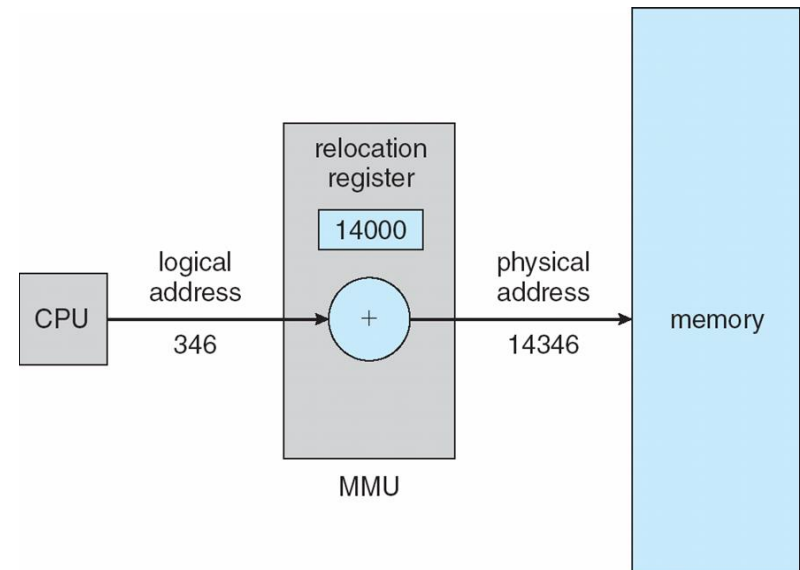
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address**
 - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter
- To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
 - Base register now called **relocation register**
 - MS-DOS on Intel 80x86 used 4 relocation registers
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
 - Execution-time binding occurs when reference is made to location in memory
 - Logical address bound to physical addresses

Dynamic relocation using a relocation register

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required



Implemented through program design

OS can help by providing libraries to implement dynamic loading

Dynamic Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- Dynamic linking – linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
 - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**
- Consider applicability to patching system libraries
 - Versioning may be needed

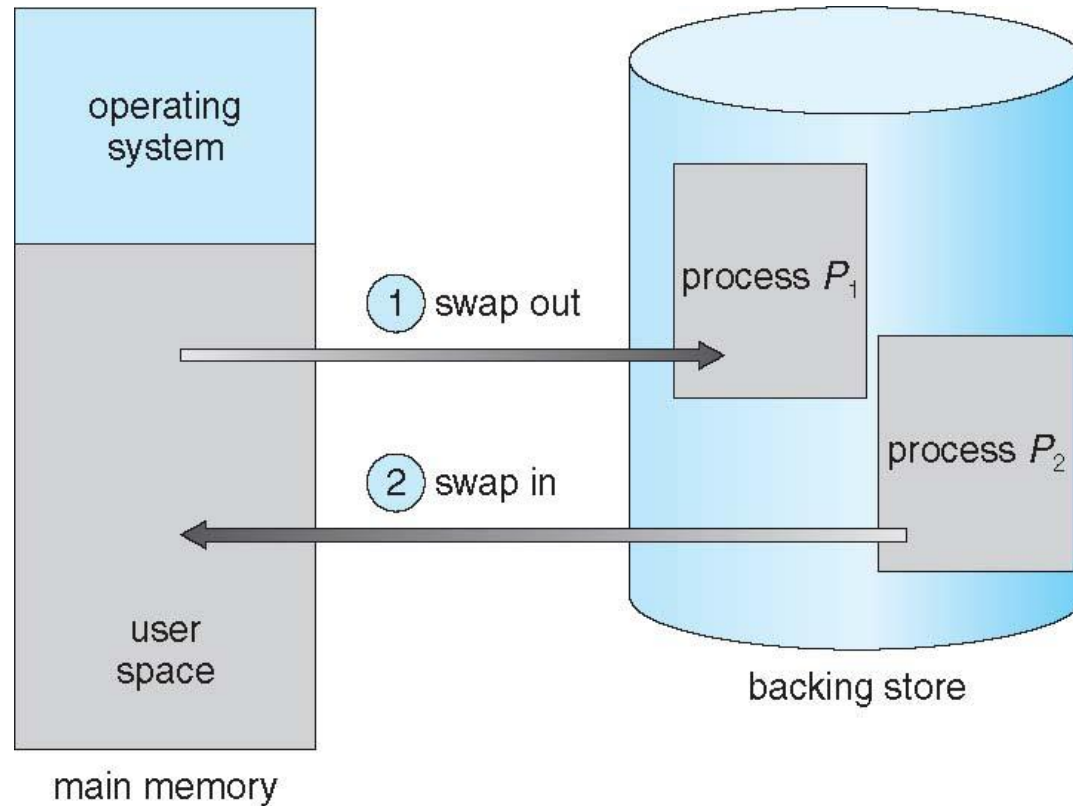
Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

Swapping (Cont.)

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
 - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
 - Swapping normally disabled
 - Started if more than threshold amount of memory allocated
 - Disabled again once memory demand reduced below threshold

Schematic View of Swapping



Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - Swap out time of 2000 ms
 - Plus swap in of same sized process
 - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
 - System calls to inform OS of memory use via `request_memory()` and `release_memory()`

Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
 - Pending I/O – can't swap out as I/O would occur to wrong process
 - Or always transfer I/O to kernel space, then to I/O device
 - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
 - But modified version common
 - Swap only when free memory extremely low

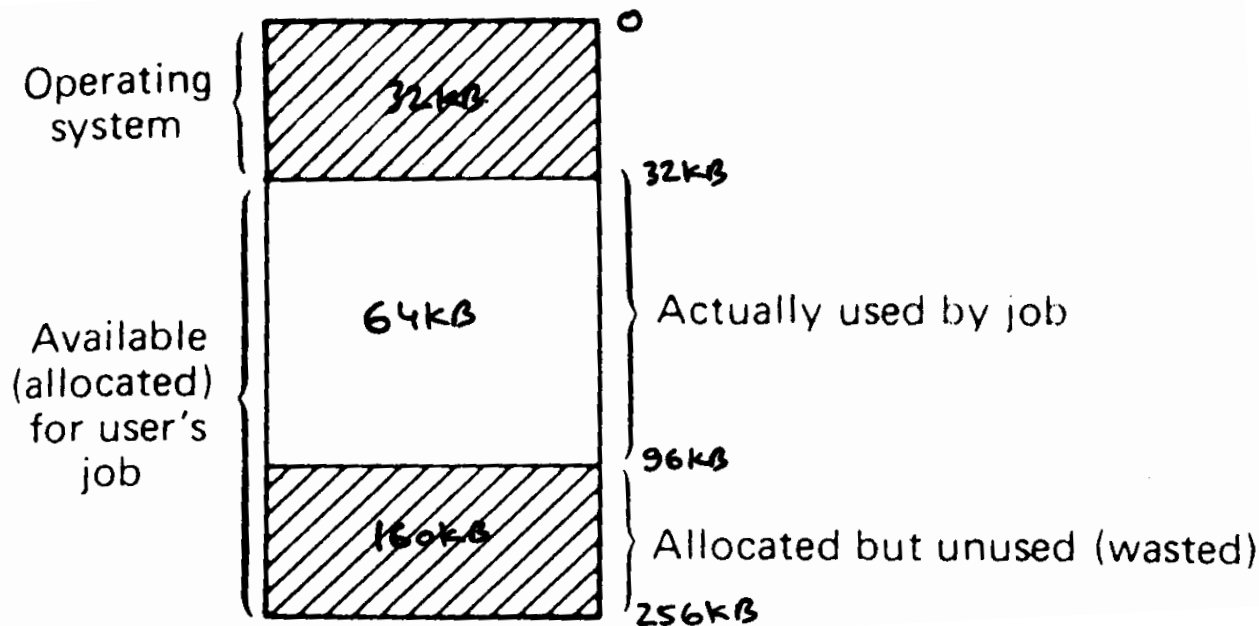
Memory Management Techniques

- Single Contiguous Memory Management
- Partitioned Memory Management
- Relocation Partitioned Memory Management
- Paged Memory Management
- Segmented Memory Management
- Demand–Paged Memory Management
 - Page Replacement Algorithms
- Overlay Memory Management

Single Contiguous Memory Management

Main memory usually into two partitions:

- Resident operating system, usually held in low memory with interrupt vector
- User processes then held in high memory
- Each process contained in single contiguous section of memory



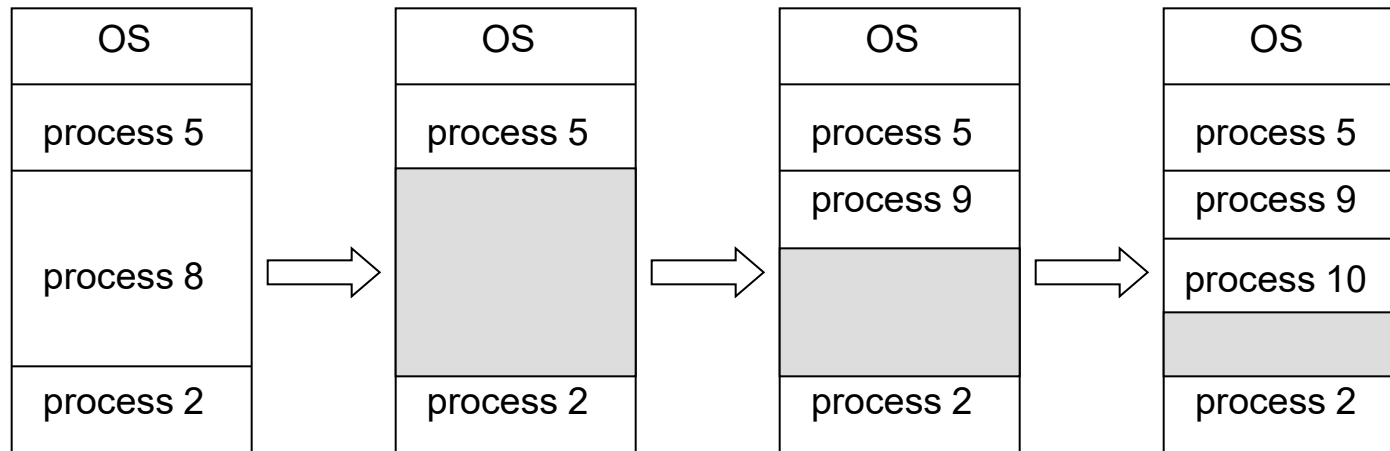
Advantages:

- Simplicity
- Available memory fully not utilised
- Limited Job Size (< Available Memory)

Contiguous Allocation (Cont.)

- **Multiple-partition allocation**

- Degree of multiprogramming limited by number of partitions
- Hole – block of available memory; holes of various size are scattered throughout memory
- When a process arrives, it is allocated memory from a hole large enough to accommodate it
- Process exiting frees its partition, adjacent free partitions combined
- Operating system maintains information about:
 - a) allocated partitions
 - b) free partitions (hole)



Partitioned Memory Management

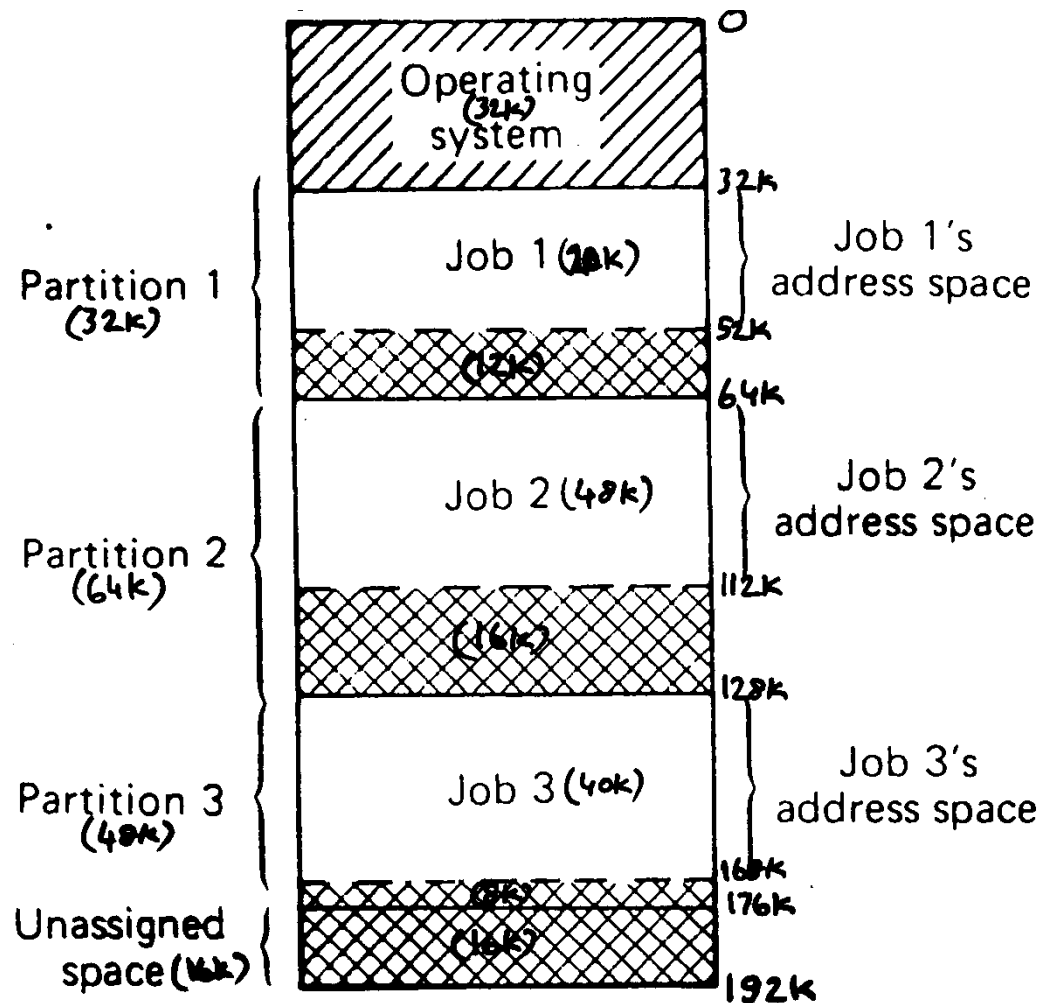


Figure 3.1 Partitioned allocation

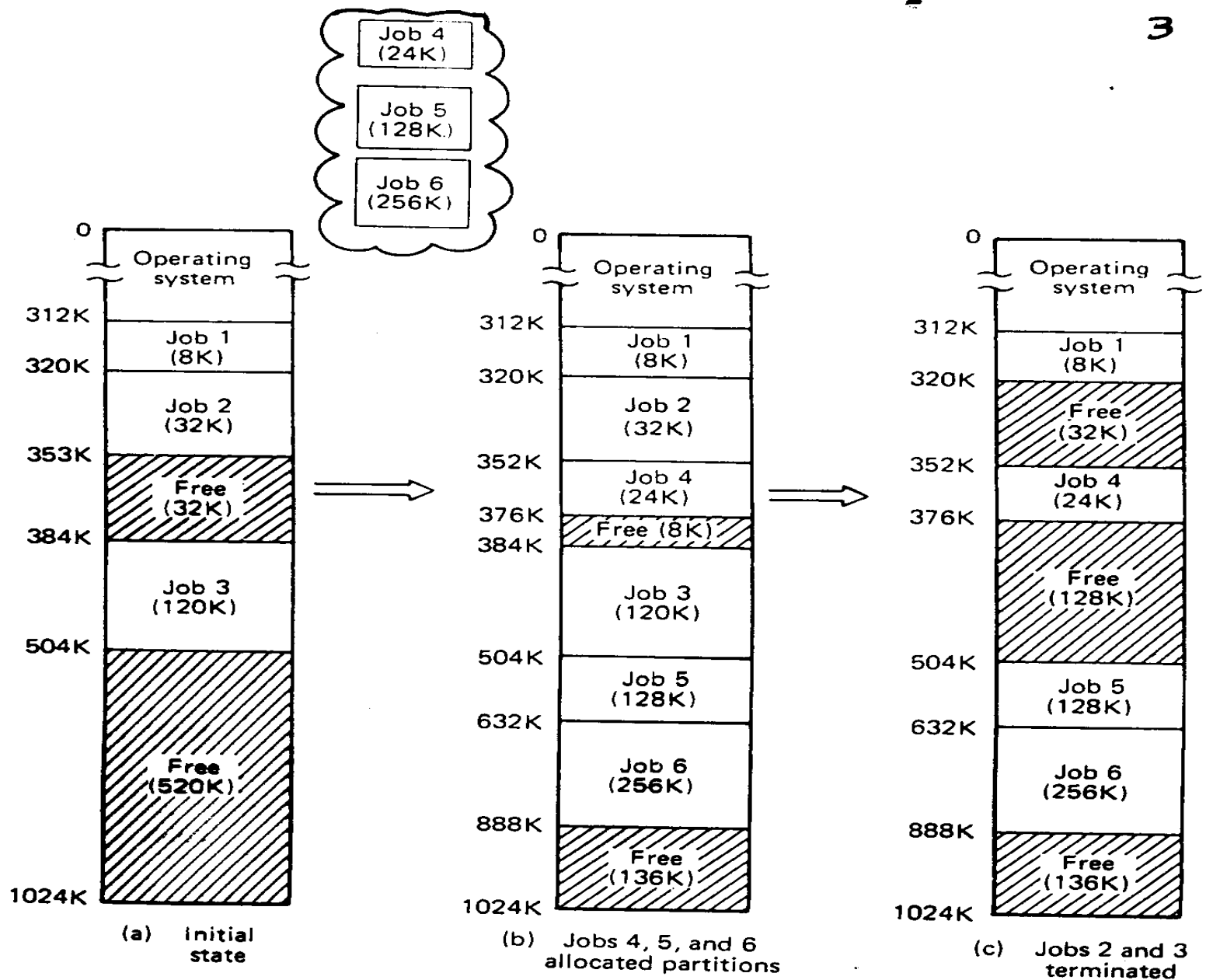


Figure 3.22 Partition allocation and deallocation

Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes

- **First-fit:**
 - Allocate the *first* hole that is big enough
- **Best-fit:**
 - Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
- **Worst-fit:**
 - Allocate the *largest* hole; must also search entire list
 - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of speed and storage utilization

1. Consider a swapping system in which memory consists of the following hole sizes in memory order: 10KB, 4KB, 20KB, 18KB, 7KB, 9KB, 12KB, and 15KB. Which hole is taken for successive segment requests of 12KB, 10KB, 9KB for *first fit*? Now repeat the question for *best fit*, and *worst fit*.

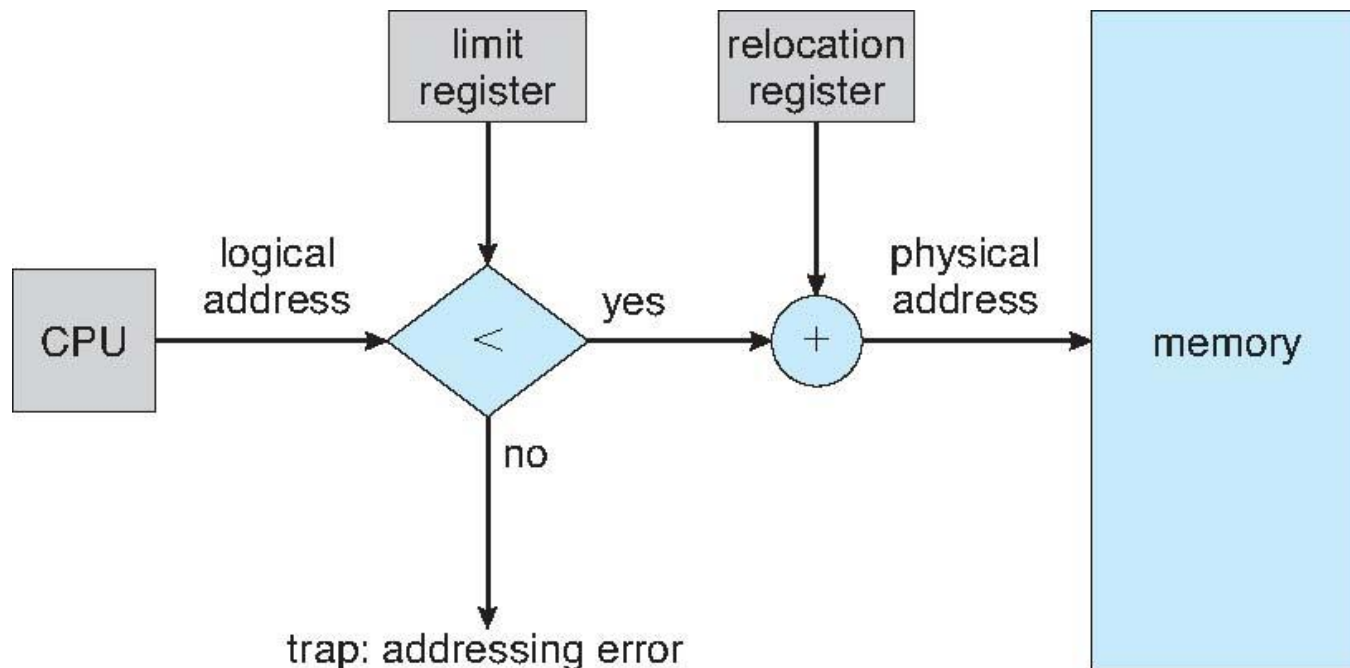
<i>Memory</i>	<i>First Fit</i>	<i>Best Fit</i>	<i>Worst Fit</i>
10 KB	10 KB (Job 2)	10 KB (Job 2)	10 KB
4KB	4KB	4KB	4KB
20 KB	12 KB (Job 1)	20 KB	12 KB (Job 1)
	8 KB		8 KB
18 KB	9 KB (Job 3)	18 KB	10 KB (Job 2)
	9 KB		8 KB
7 KB	7 KB	7 KB	7 KB
9 KB	9 KB	9 KB (Job3)	9 KB
12 KB	12 KB	12 KB (Job 1)	12 KB
15 KB	15 KB	15 KB	9 KB Job 3)
			6 KB

2. Given memory partitions of 12KB, 7KB, 15KB, 20KB, 9KB, 4KB, 10KB, and 18KB (in order), how would each of the *first-fit* and *best-fit* algorithms place processes of 10KB, 12KB, 6KB, and 9KB (in order)?

Protecting a memory segment from other processes: Hardware Support for Relocation and Limit Registers

Relocation registers used to protect user processes from each other, and from changing operating-system code and data

- Base register contains value of smallest physical address
- Limit register contains range of logical addresses – each logical address must be less than the limit register
- MMU maps logical address *dynamically*
- Can then allow actions such as kernel code being **transient** and kernel changing size



Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation
 - $1/3$ may be unusable -> **50-percent rule**

Solution to the External Fragmentation

- Reduce external fragmentation by **compaction**
 - Shuffle memory contents to place all free memory together in one large block.

Constraints:

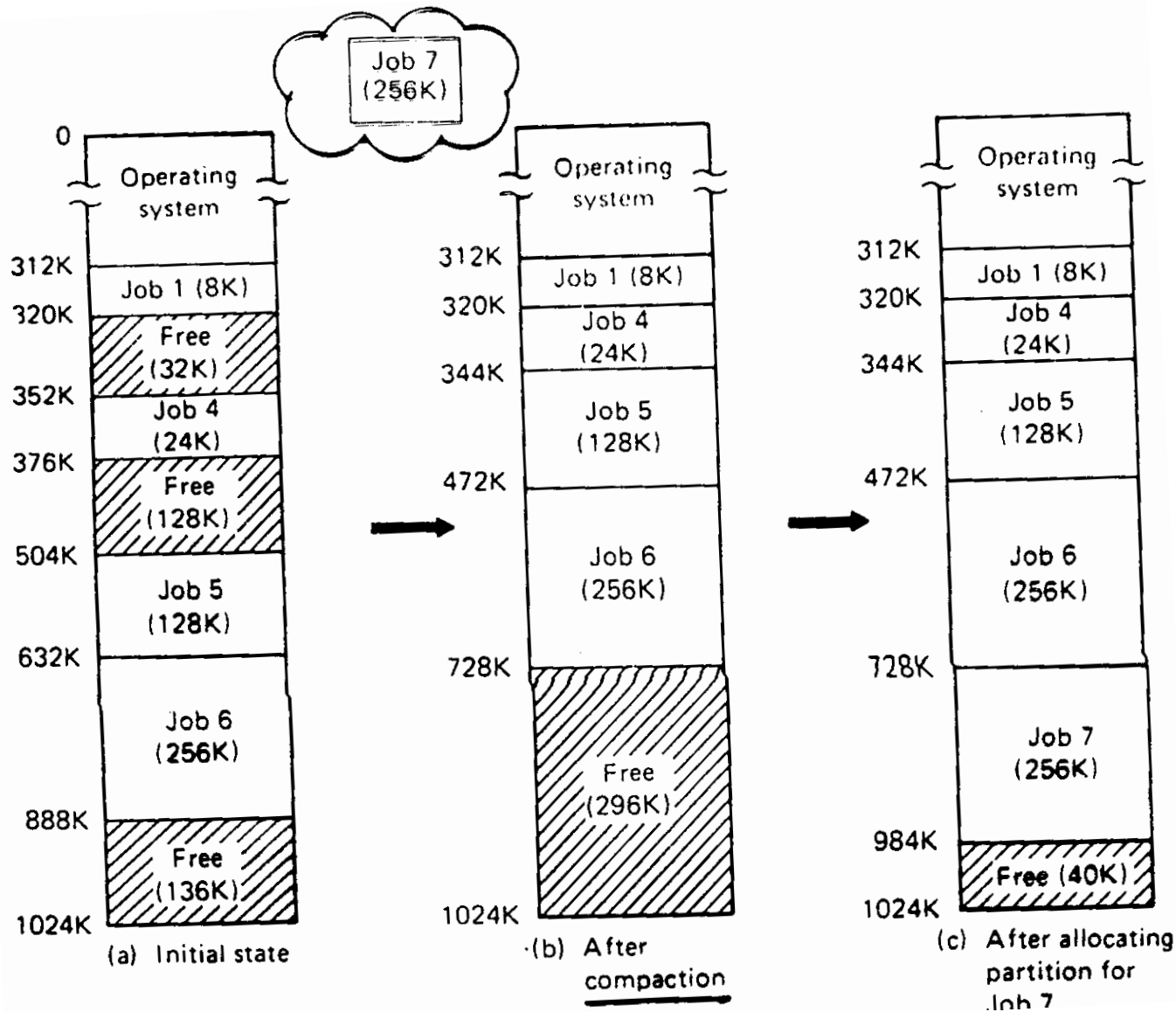
1. Compaction is possible *only* if relocation is dynamic, and is done at execution time
 2. I/O problem
 - Latch job in memory while it is involved in I/O
 - Do I/O only into OS buffers
- Now consider that backing store has same fragmentation problems

Relocation Partitioned Memory Management

Compaction / Burping / Recompaction / Reburping

- Periodically combining all free areas in between partitions into one contiguous area.
- Move the contents of all allocated partitions to become one contiguous.

Relocation Partitioned Memory Management

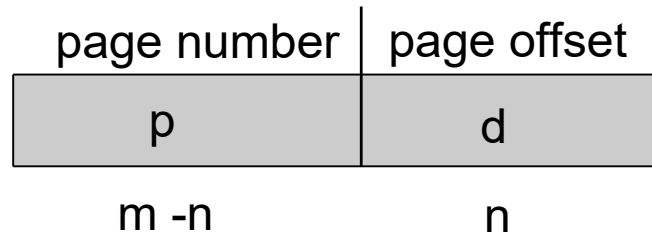


Paging

- Physical address space of a process can be noncontiguous.
- Process is allocated physical memory whenever the latter is available
- Divide physical memory into fixed-sized blocks called **frames**
(Size is power of 2, between 512 bytes and 8,192 bytes)
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size n pages, need to find n free frames and load program
- Set up a page table to translate logical to physical addresses
- Internal fragmentation

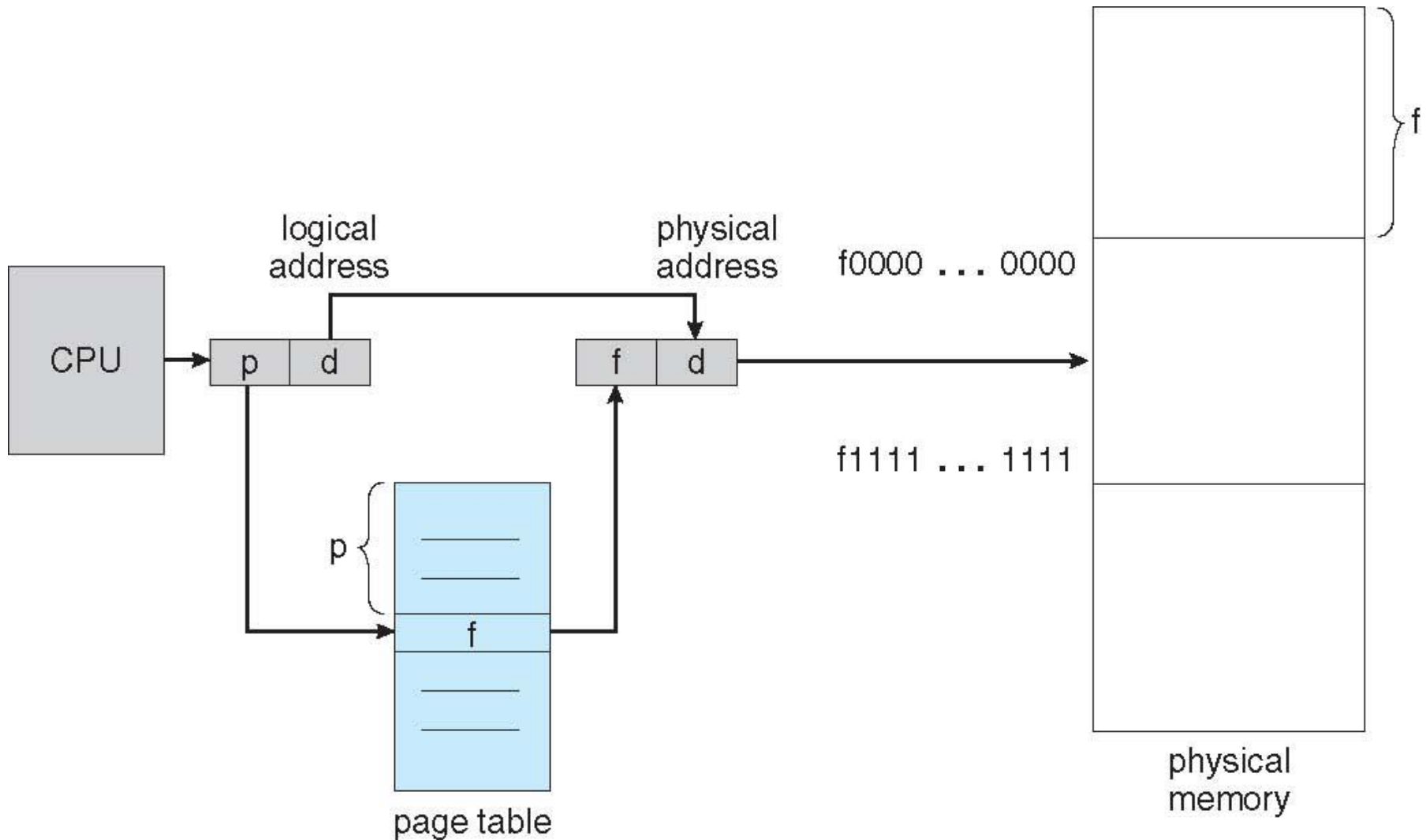
Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit

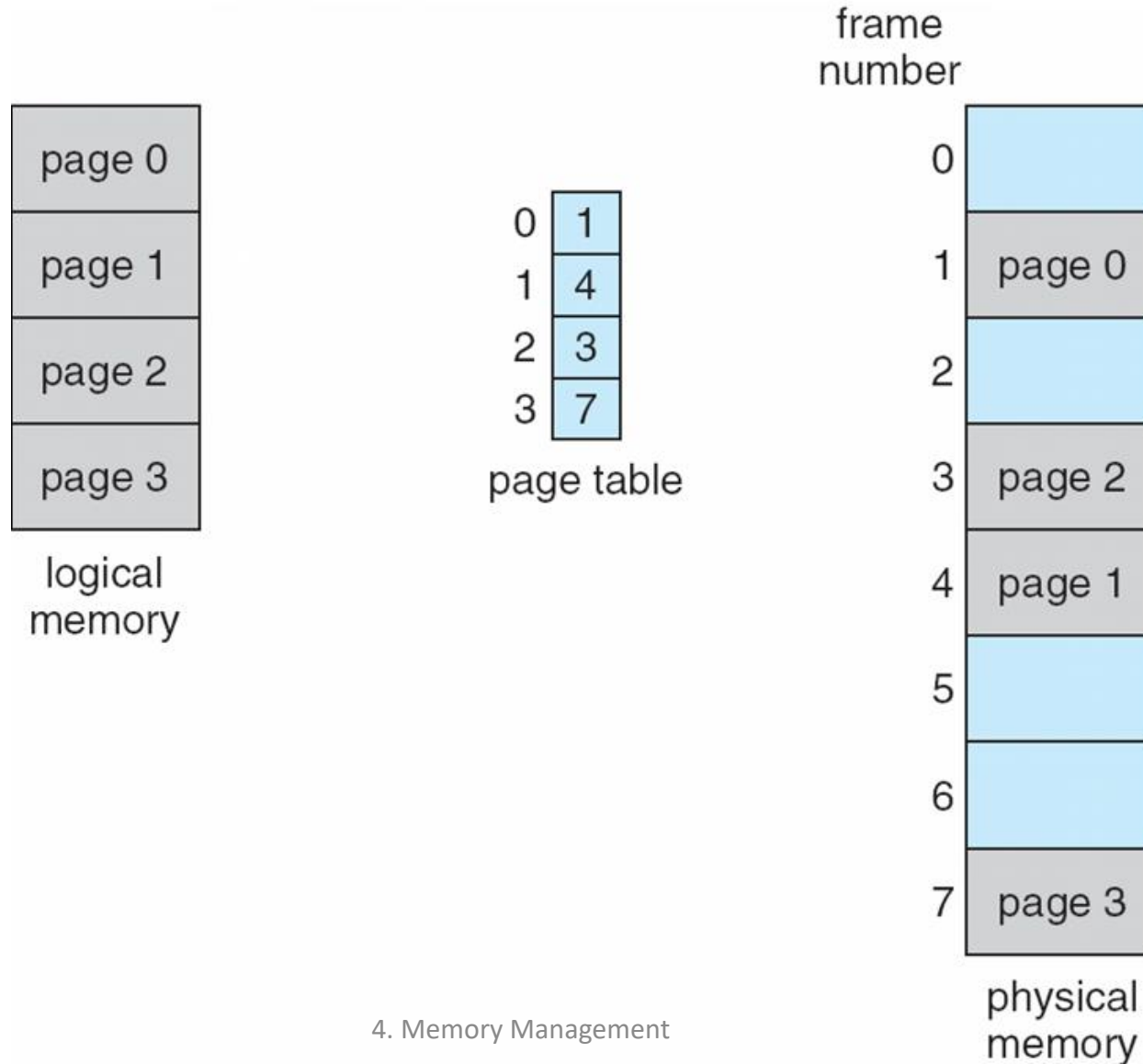


- For given logical address space 2^m and page size 2^n

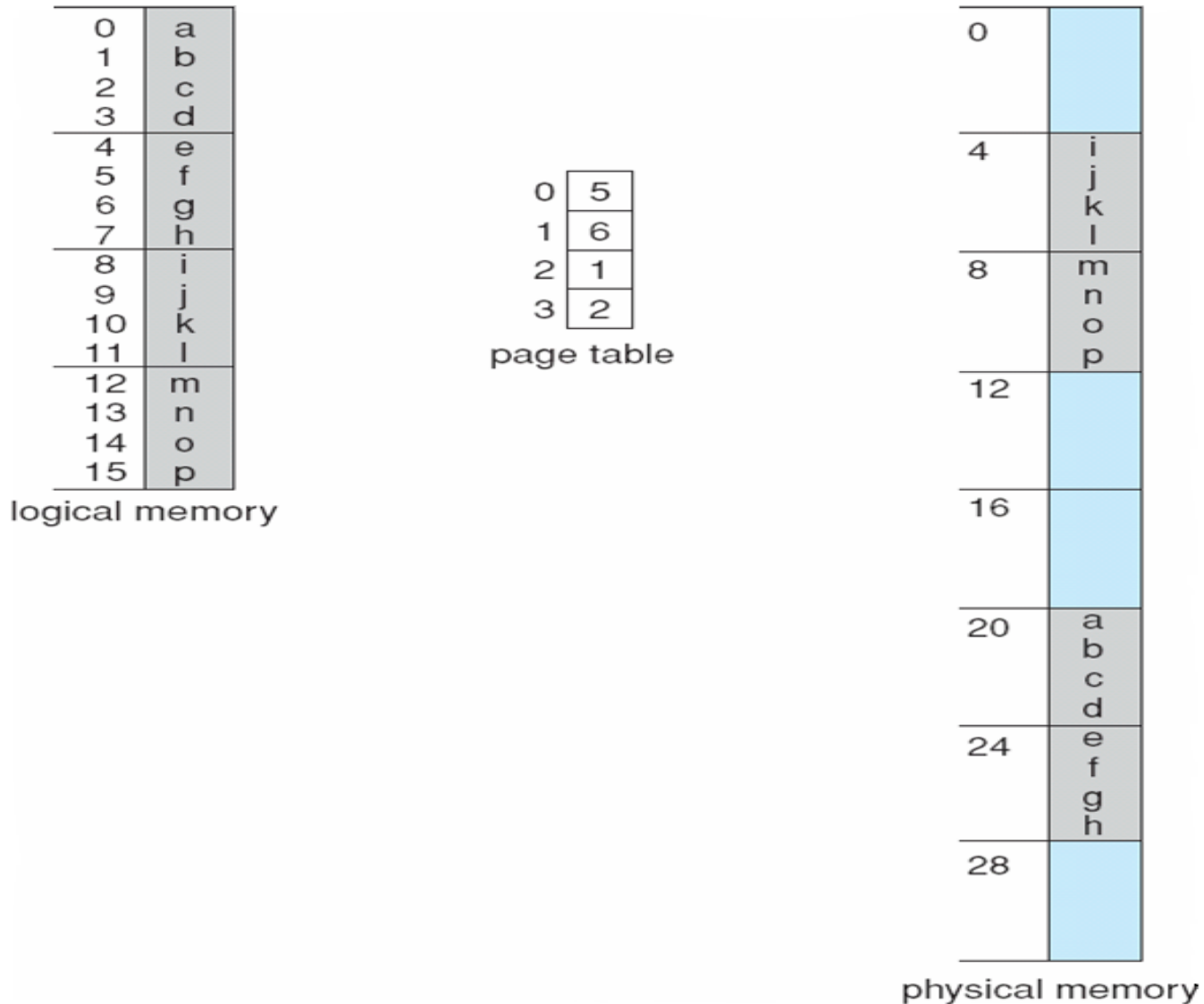
Paging Hardware



Paging Model of Logical and Physical Memory



Paging Example

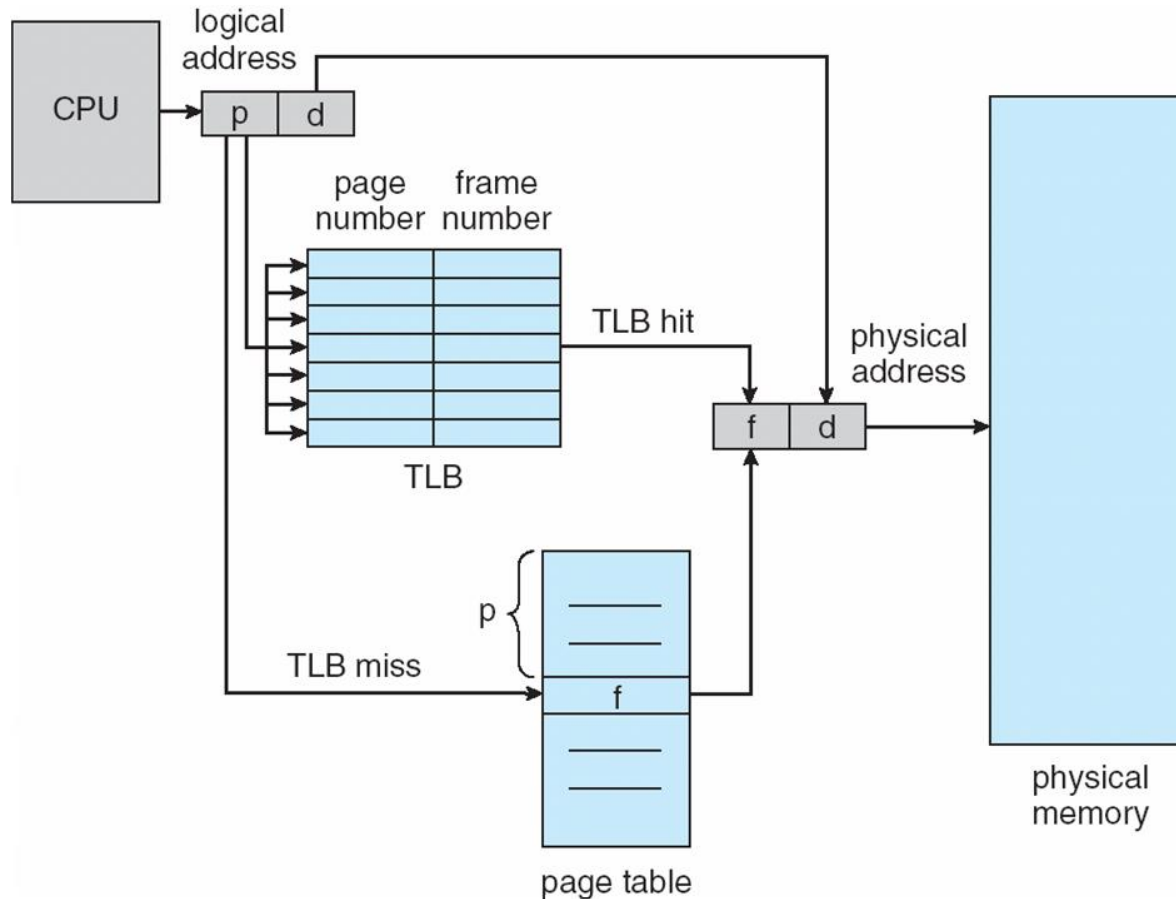


$n=2$ and $m=4$ 32-byte memory and 4-byte pages

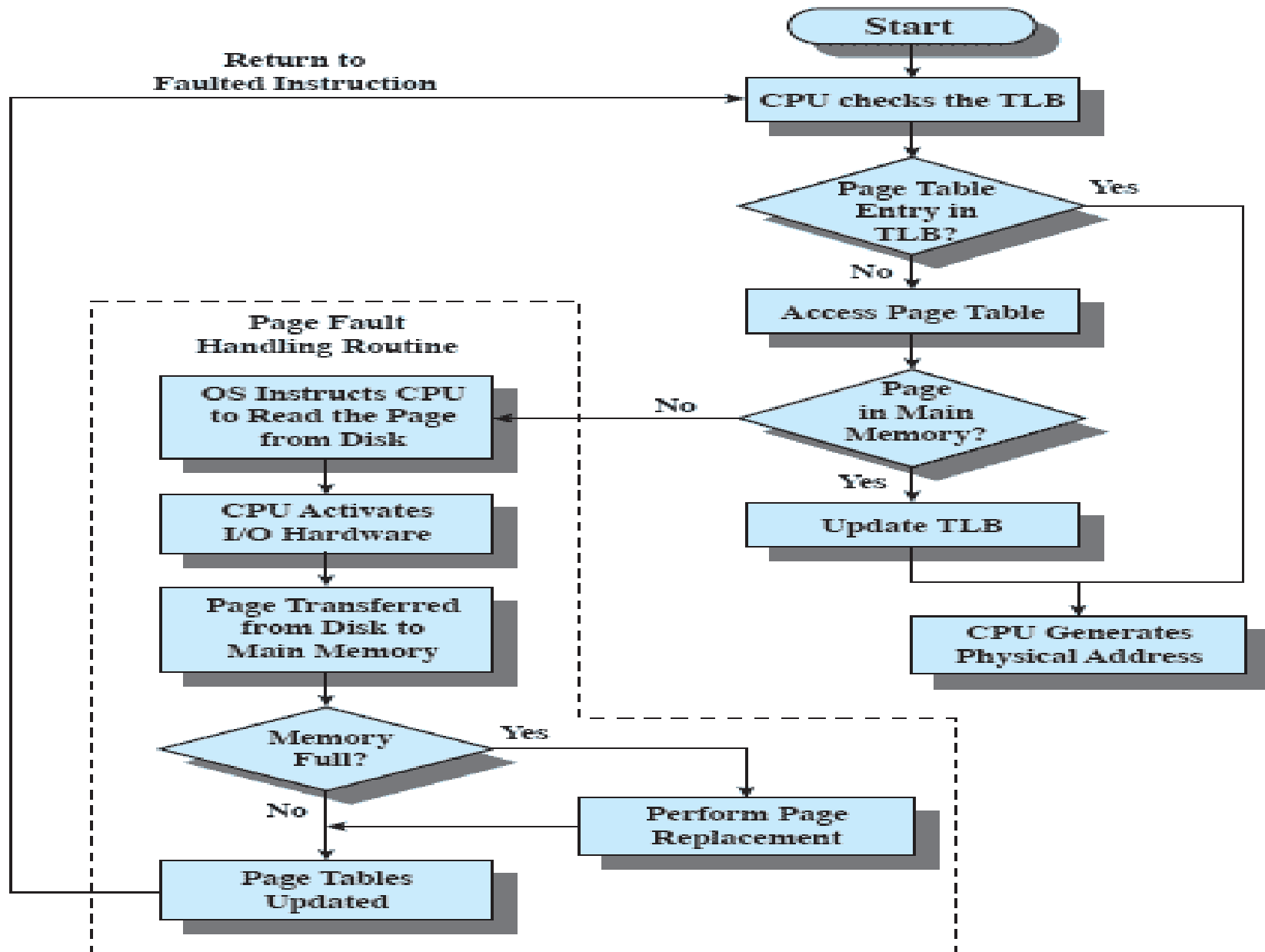
Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

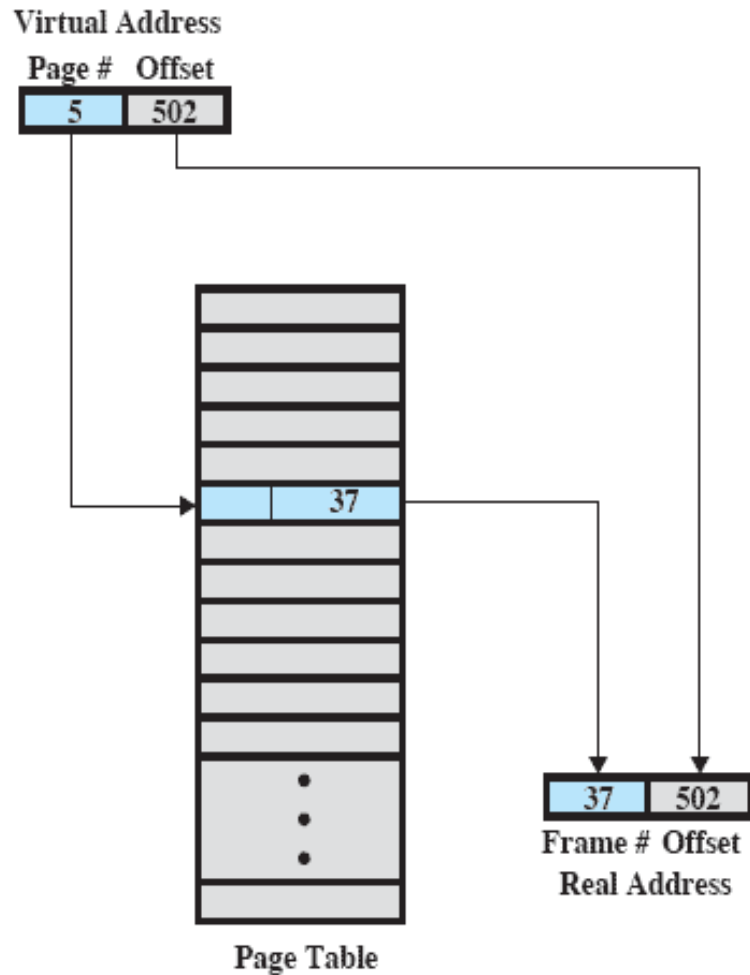
Paging Hardware With TLB



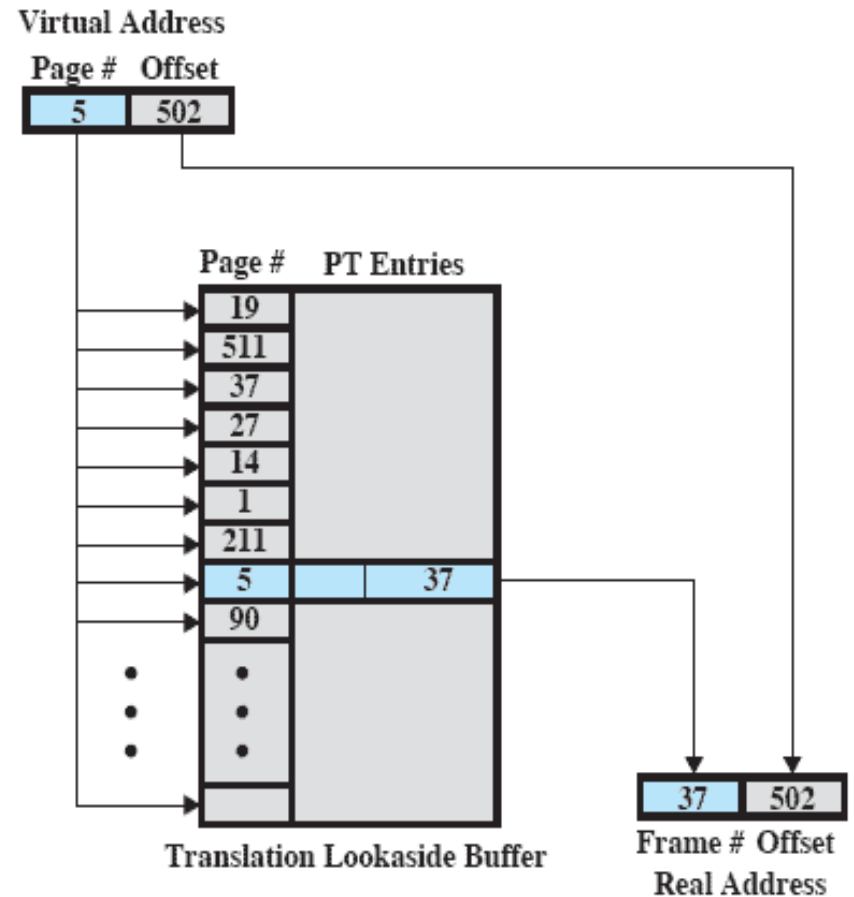
Operation of Paging and TLB



Direct Vs. Associative Lookup for Page Table Entries using TLB



(a) Direct mapping

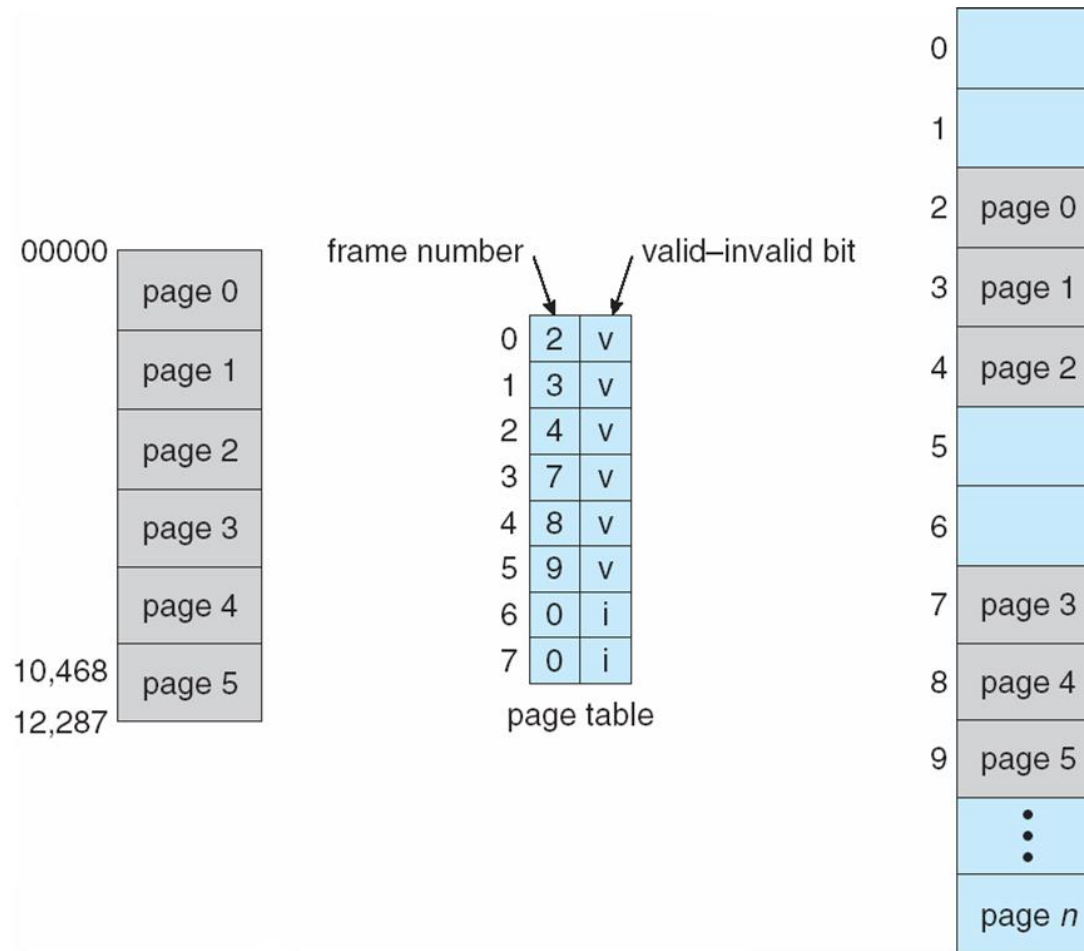


(b) Associative mapping

Memory Protection

- Memory protection implemented – by associating protection bit with each frame
 - to indicate if read-only or read-write access is allowed
 - Can also add more bits to indicate page execute-only, and so on
- Valid-invalid bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use page-table length register (PTLR)
 - Page Table Entries (PTEs) can contain more information
- Any violations result in a trap to the kernel

Valid (v) or Invalid (i) Bit In A Page Table



Shared Pages

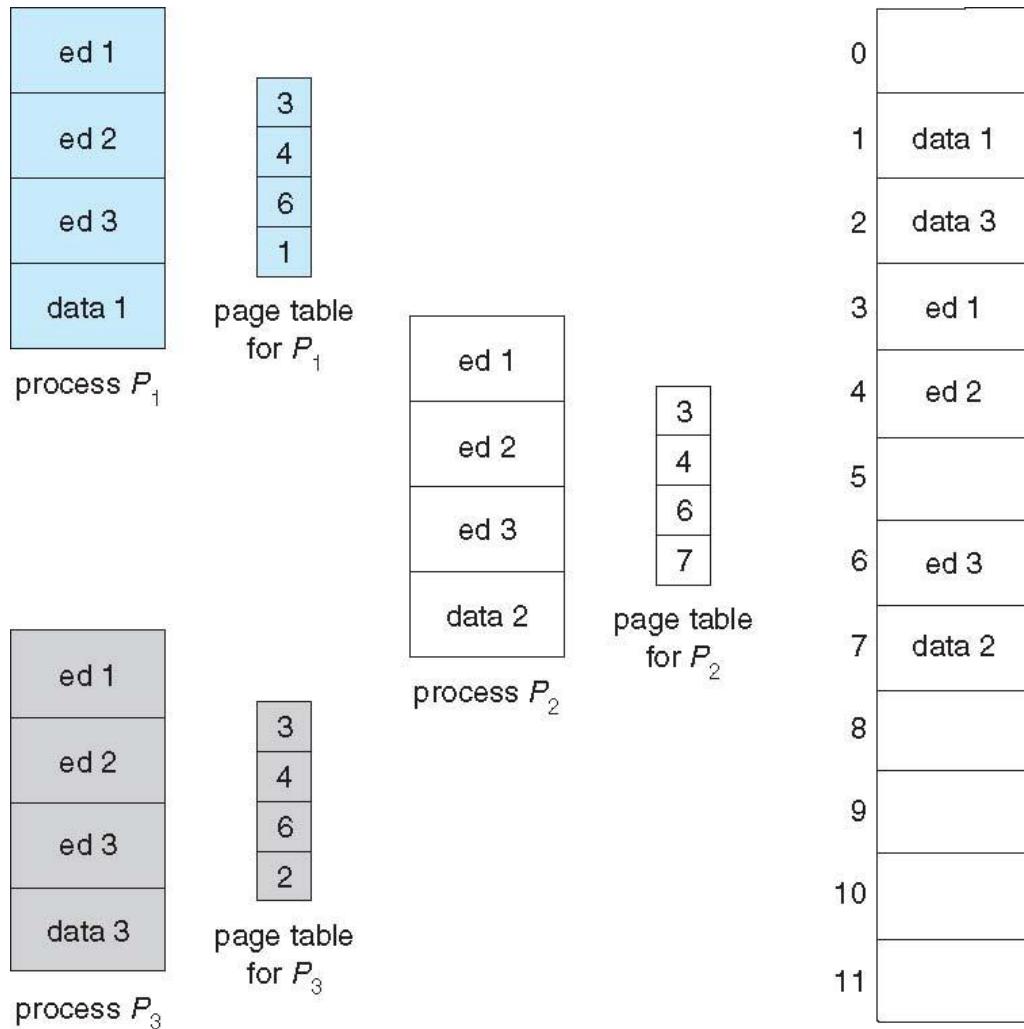
- **Shared code**

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

- **Private code and data**

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example



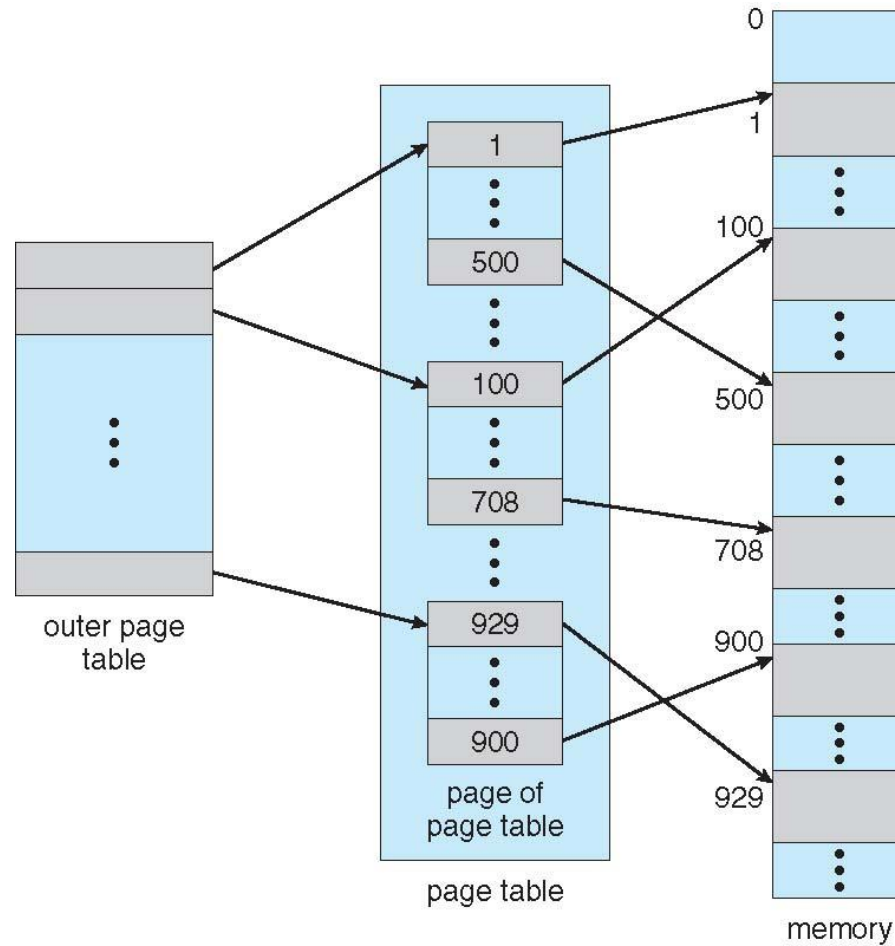
Structure of the Page Table

- Memory structures for paging can get huge using straightforward methods
 - Consider a 32-bit logical address space as on modern computers
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes $\rightarrow$ 4 MB of physical address space / memory for page table alone
 - That amount of memory used to cost a lot
 - Don't want to allocate that contiguously in main memory
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table

Two-Level Page-Table Scheme



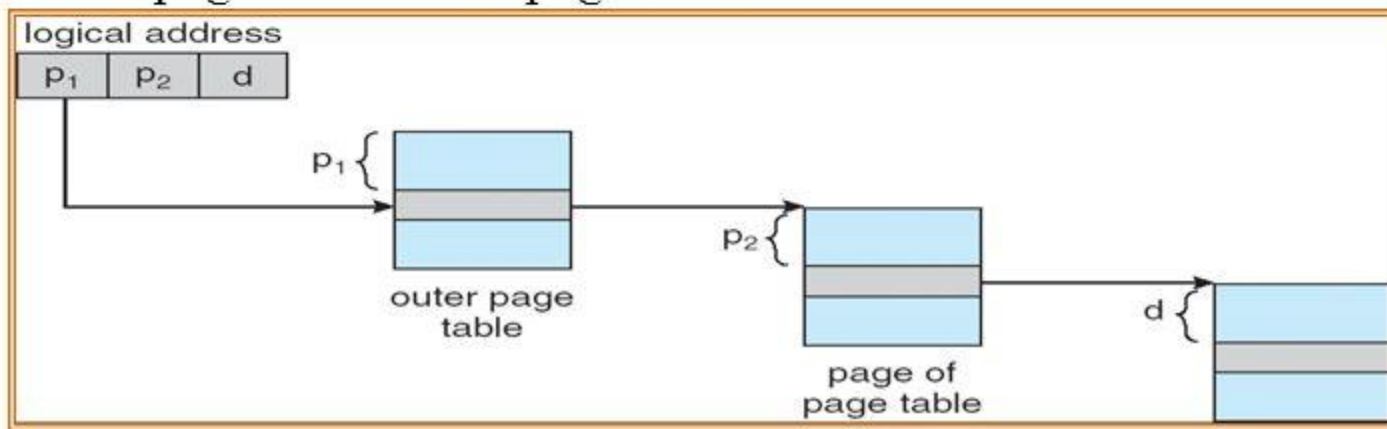
Two-level paging example

- A logical address (on 32-bit machine with 4K page size) is divided into:
 - a page number consisting of 20 bits
 - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
 - a 10-bit page number
 - a 12-bit page offset

page number		page offset
p_1	p_2	d
10	10	12

Thus, a logical address is as follows:

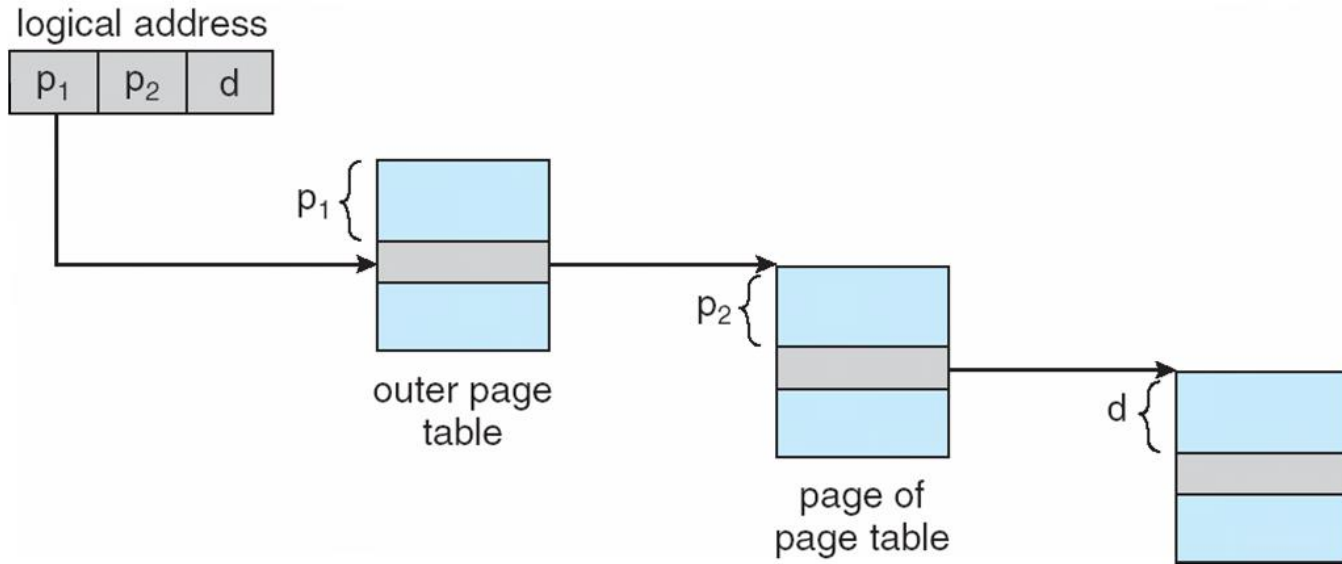
where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the outer page table



where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table

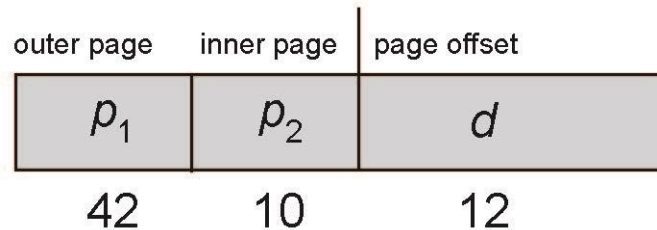
Known as **forward-mapped page table**

Address-Translation Scheme



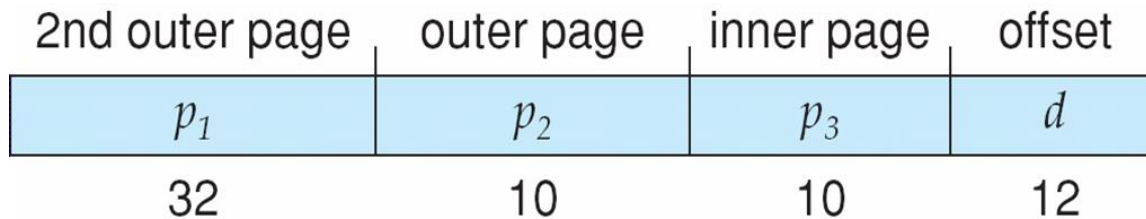
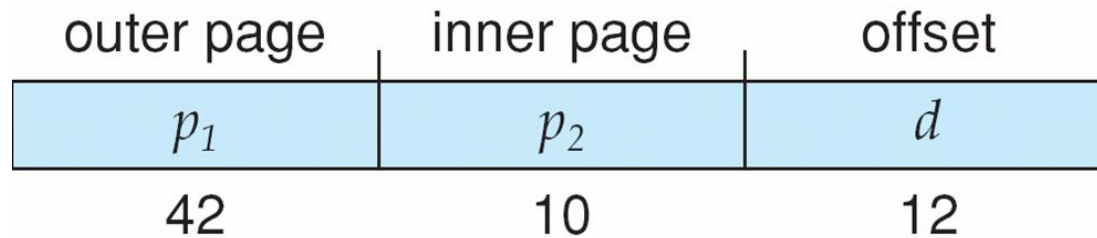
64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2nd outer page table
- But in the following example the 2nd outer page table is still 2^{34} bytes in size
 - And possibly 4 memory access to get to one physical memory location

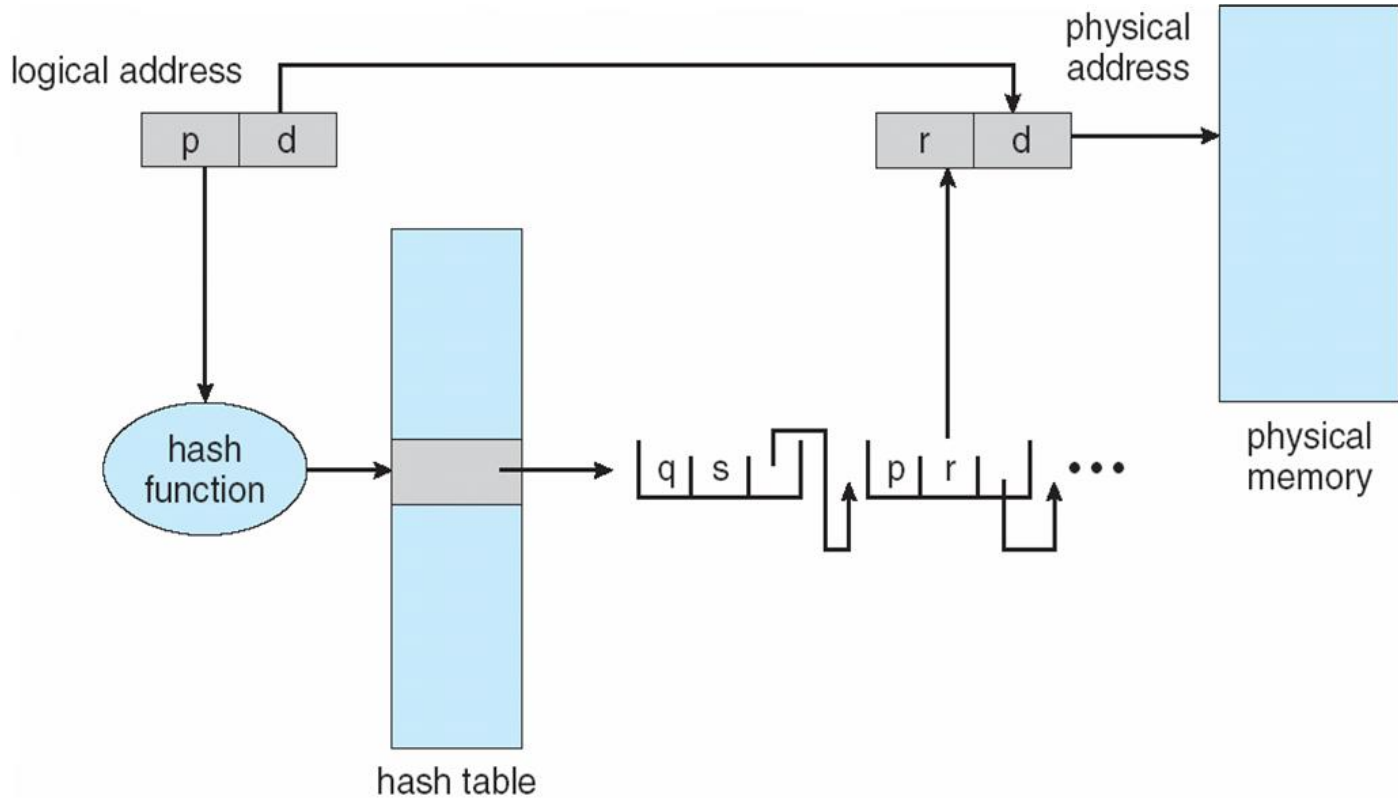
Three-level Paging Scheme



Hashed Page Tables

- Common in address spaces > 32 bits
- The virtual page number is hashed into a page table
 - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
 - If a match is found, the corresponding physical frame is extracted
- Variation for 64-bit addresses is **clustered page tables**
 - Similar to hashed but each entry refers to several pages (such as 16) rather than 1
 - Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)

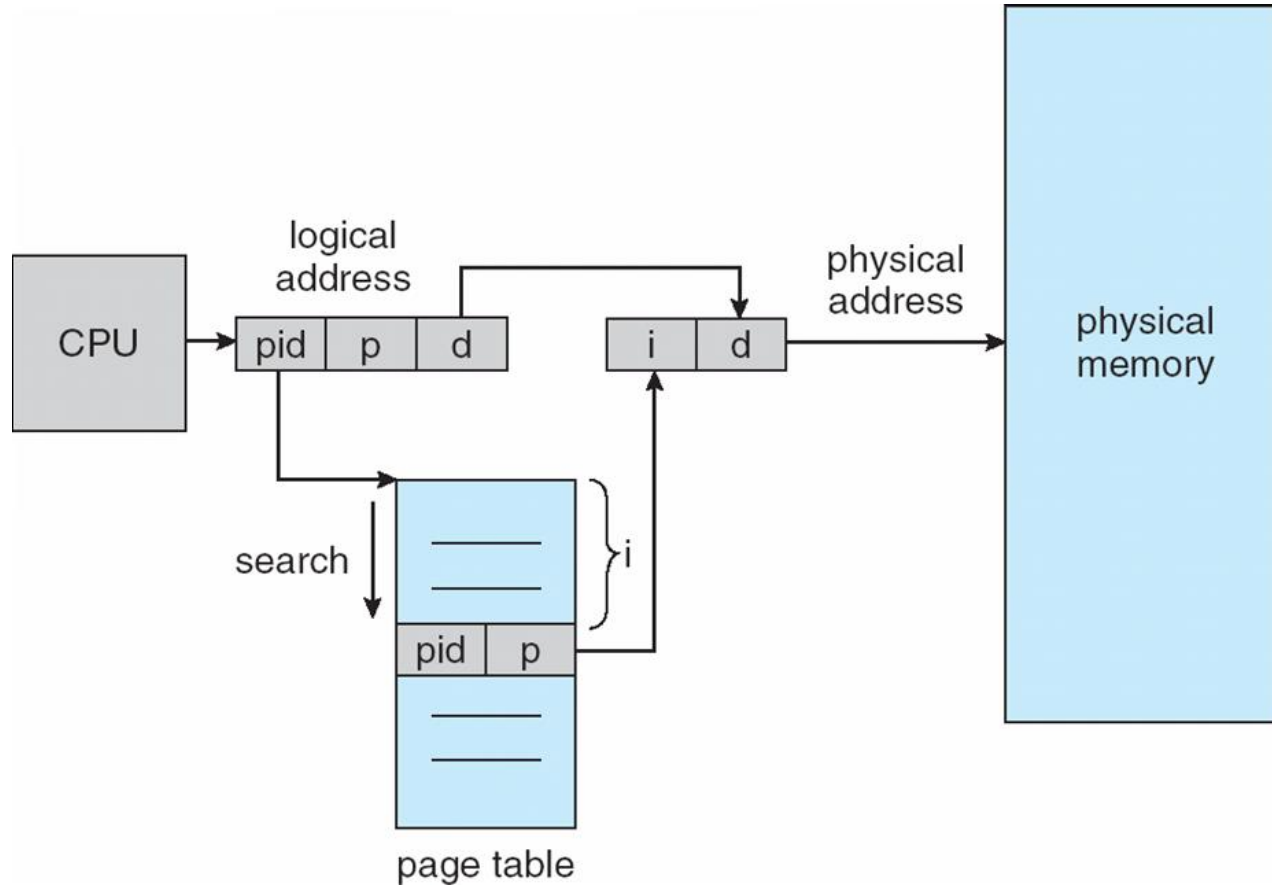
Hashed Page Table



Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
 - TLB can accelerate access
- But how to implement shared memory?
 - One mapping of a virtual address to the shared physical address

Inverted Page Table Architecture



Segmentation

- *Program*

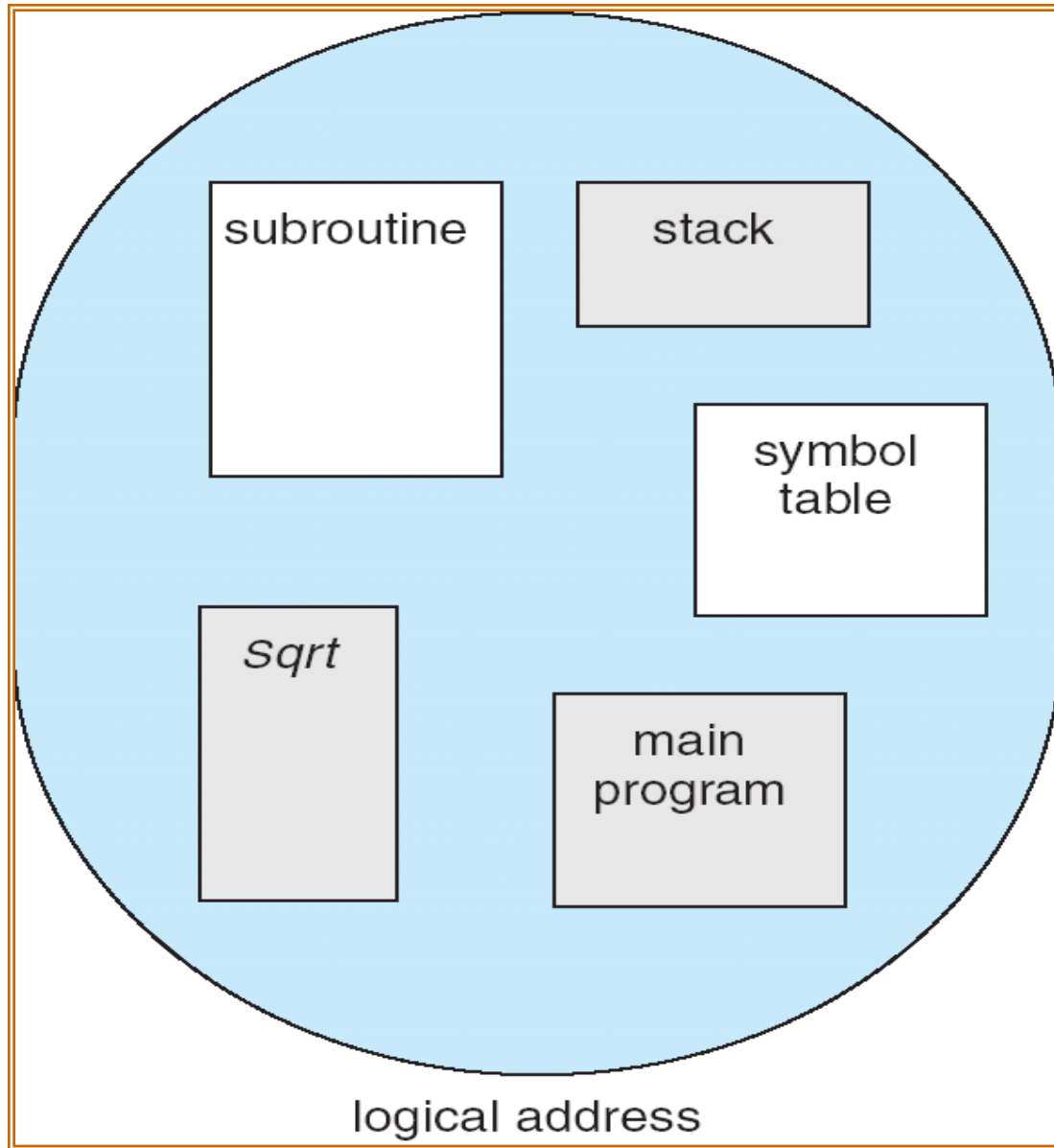
Collection of Segments

- *Segment*

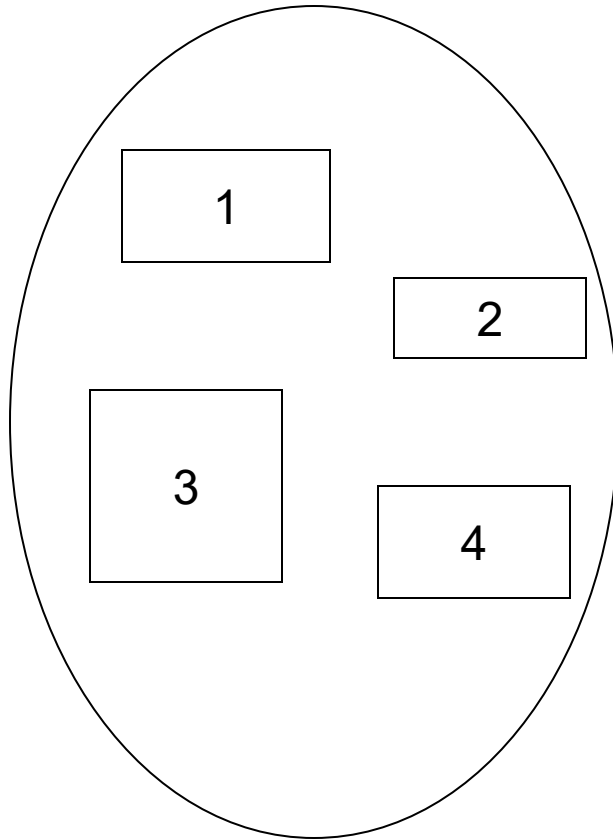
Logical Unit

- Main Program
- Procedure
- Function
- Method
- Object
- Local / Global Variables
- Common Block
- Stack
- Symbol Table
- Arrays

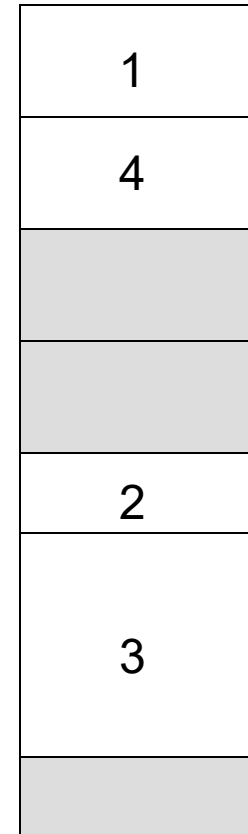
User's View of a Program



Logical View of Segmentation



user space



physical memory space

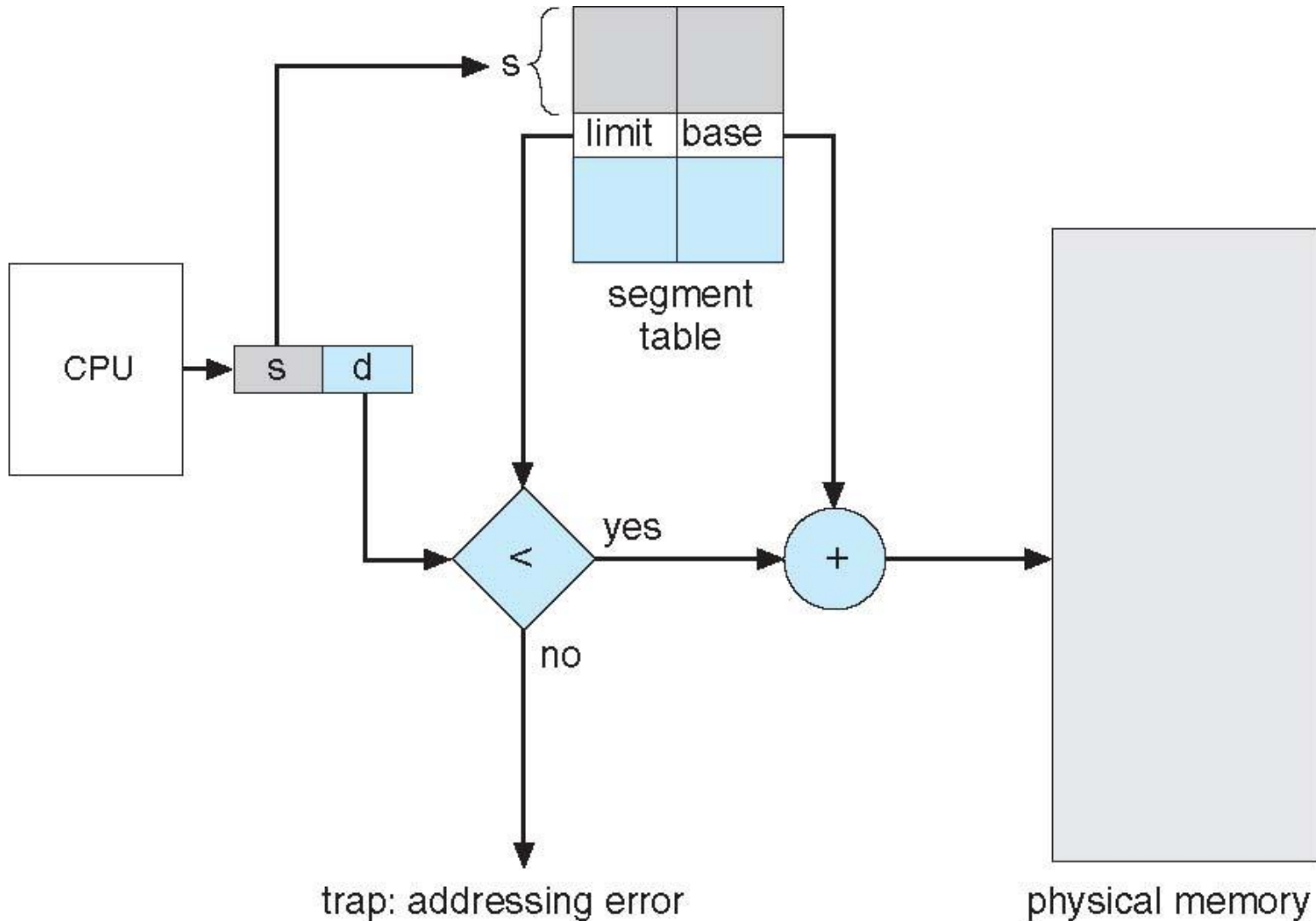
Segmentation Architecture

- Logical address consists of a two tuple:
 <segment-number, offset>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - **base** – contains the starting physical address where the segments reside in memory
 - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
 segment number **s** is legal if **s** < **STLR**

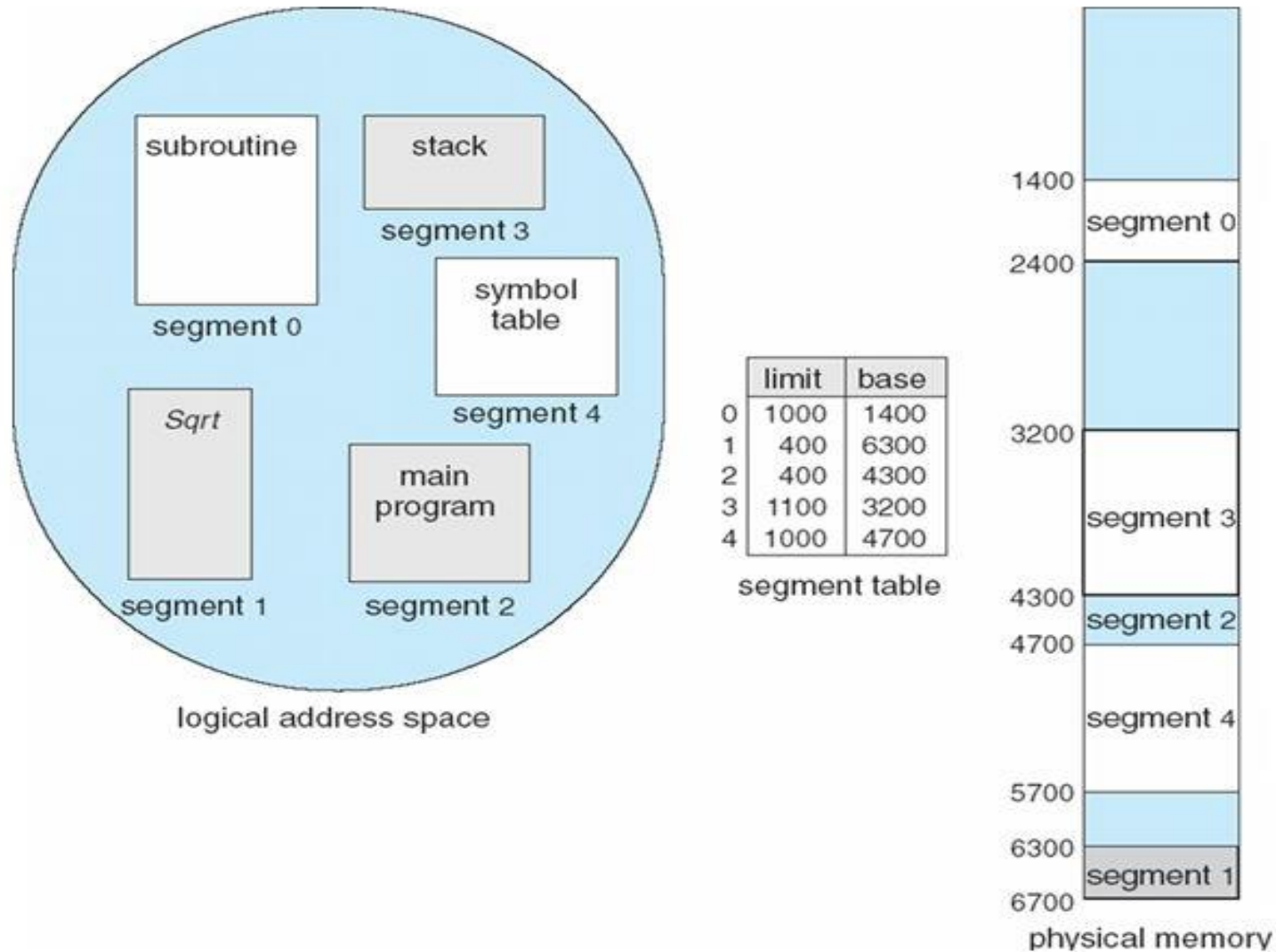
Segmentation Architecture (Cont.)

- Protection
 - With each entry in segment table associate:
 - validation bit = 0 $\Rightarrow$ illegal segment
 - read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem
- A segmentation example is shown in the following diagram

Segmentation Hardware



Example of Segmentation



Contd..

- Here we have five segments numbered from 0 through 4.
- The segments are stored in physical memory as shown.
- The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory(or base) and the length of that segment(or limit).
- For example, segment 2 is 400 bytes long and begins at location 4300. Thus, a reference to byte 53 of segment 2 is mapped onto location $4300+53=4353$.
- A reference to byte 1222 of segment 0 would result in a trap to the operating system, as this segment is only 1,000 bytes long.

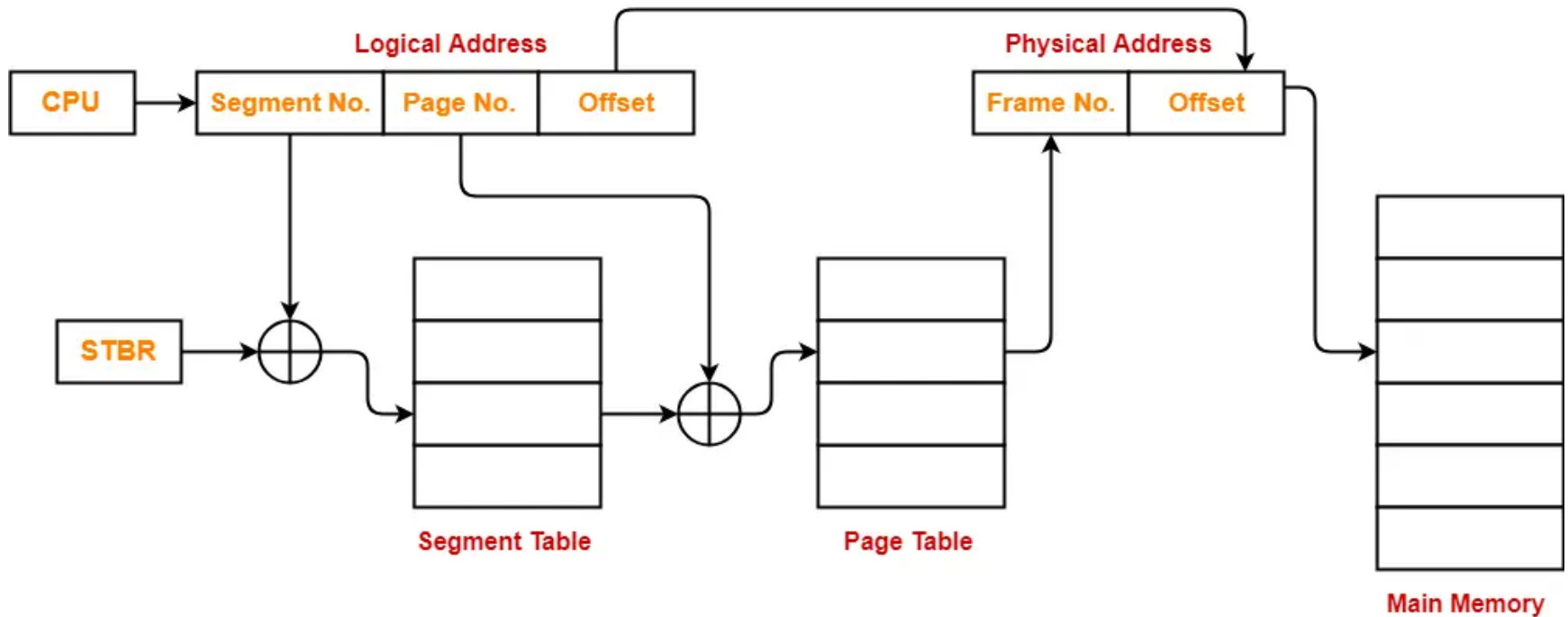
Difference between Paging and Segmentation

BASIS FOR COMPARISON	PAGING	SEGMENTATION
Basic	A page is of fixed block size.	A segment is of variable size.
Fragmentation	Paging may lead to internal fragmentation.	Segmentation may lead to external fragmentation.
Address	The user specified address is divided by CPU into a page number and offset.	The user specifies each address by two quantities a segment number and the offset (Segment limit).
Size	The hardware decides the page size.	The segment size is specified by the user.
Table	Paging involves a page table that contains base address of each page.	Segmentation involves the segment table that contains segment number and offset (segment length).

Segmentation with Paging

- Most of the architecture support paging and segmentation. All the pages of segment need not be in main memory.
- • It simplify the memory allocation and speed increases. It requires a high speed register to store the base address of the segment map table.
- • For each segment, page table is created by OS. A pointer to the page table is kept in the segment's entry in the segment table.
- • Segments are typically larger than pages.
- • The base address get from the segment descriptor table is concatenated with the offset. This new address is referred to as a linear address. Linear address is generated by paging hardware.
- **Advantage**
- Combines all advantages of paging and segmentation.
- **Disadvantages**
- 1. It increases hardware cost.
- 2. It increases processor overheads.

Segmentation with Paging



Translating Logical Address into Physical Address

Virtual Memory (VM)– Background

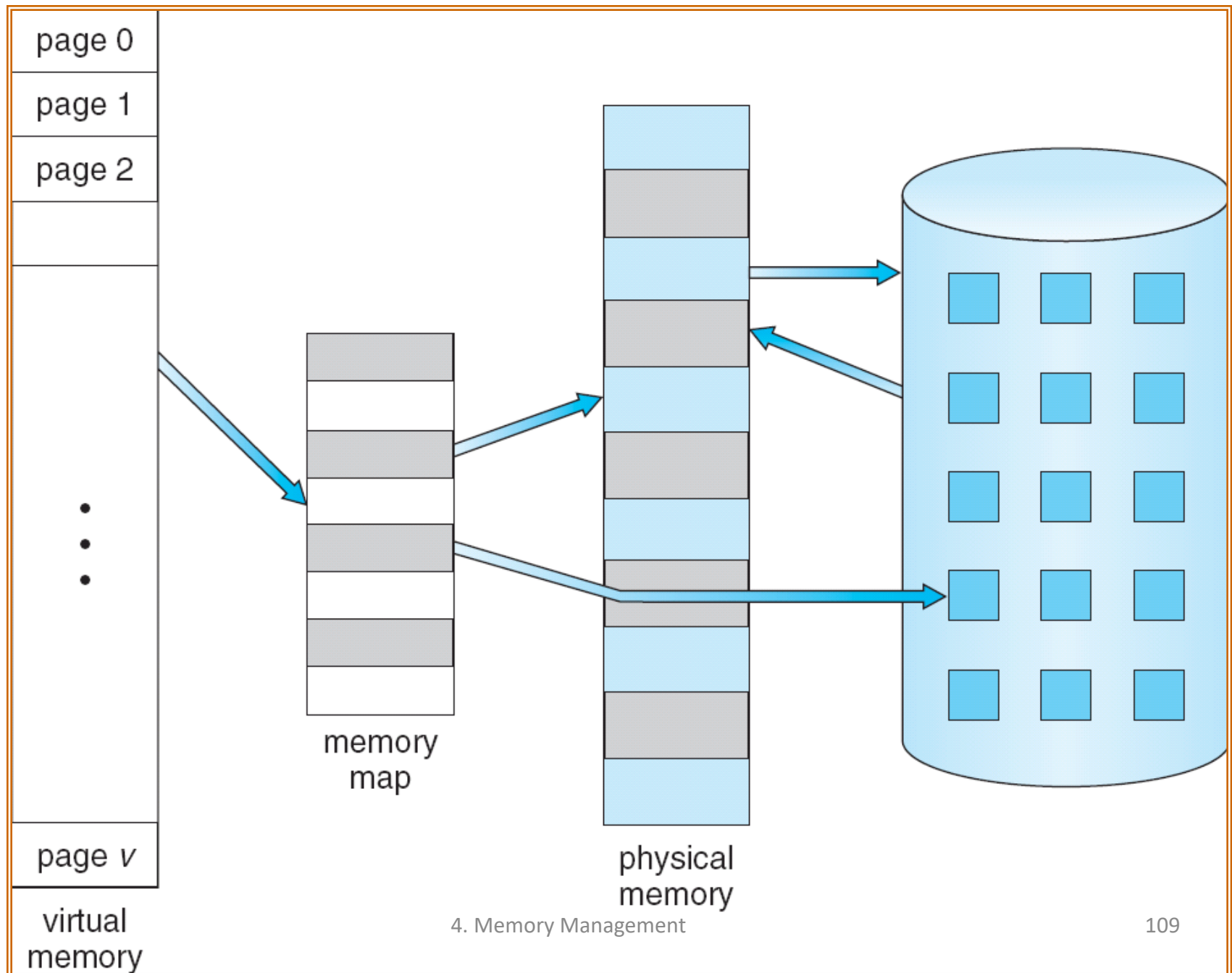
- **Virtual memory –**

It's a technique that allows the execution of processes that are not completely in memory.

Advantage of using VM:

1. programs can be larger than physical memory.
 2. Virtual memory abstracts MM into an extremely large, uniform array of storage, separating user logical memory from physical memory.
- Only part of the program needs to be in memory for execution
 - Logical address space can therefore be much larger than physical address space
 - Allows address spaces to be shared by several processes
 - Allows for more efficient process creation
- Virtual memory can be implemented via:
 - Demand paging
 - Demand segmentation

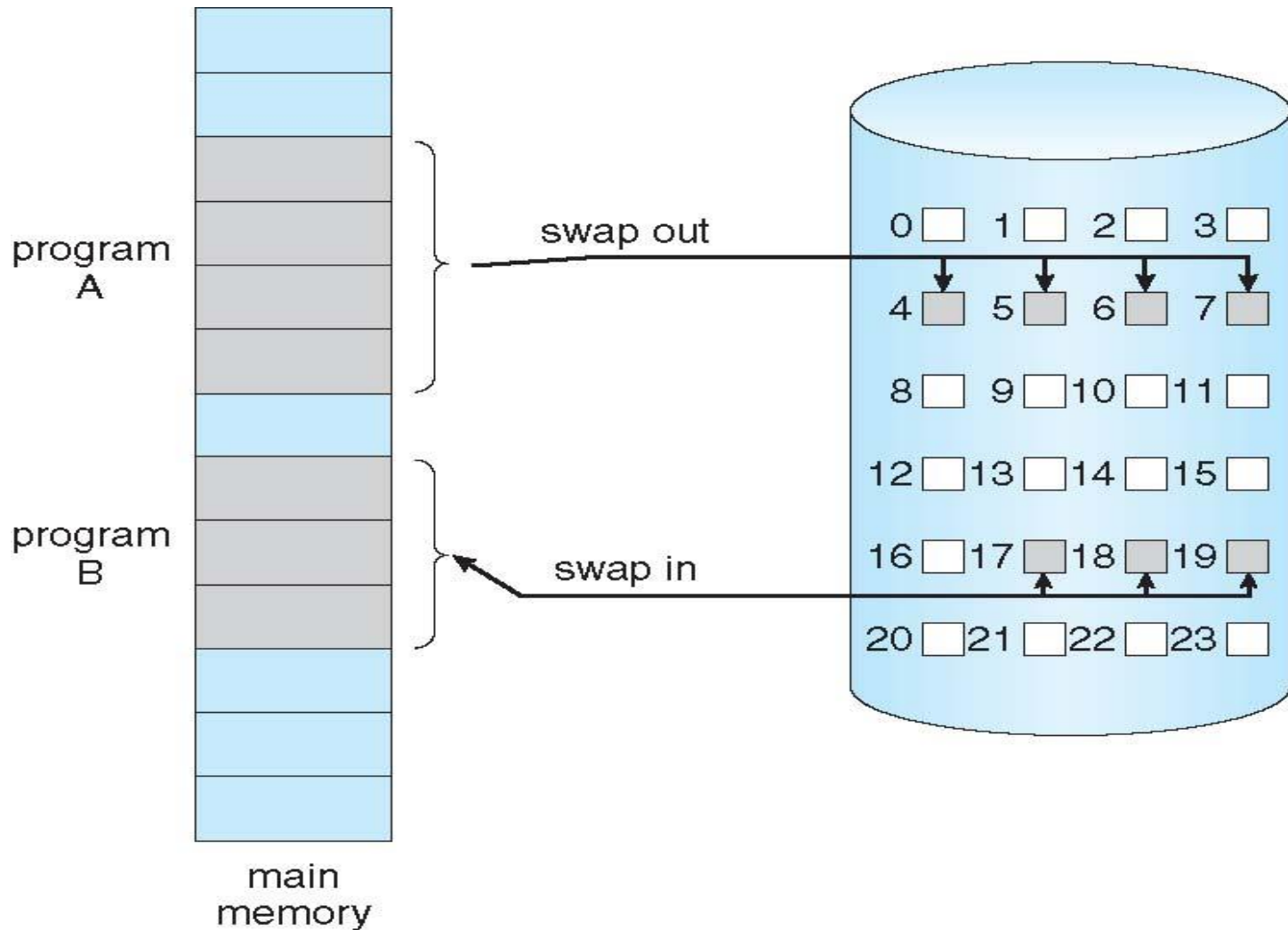
Virtual Memory That is Larger Than Physical Memory



Demand Paging

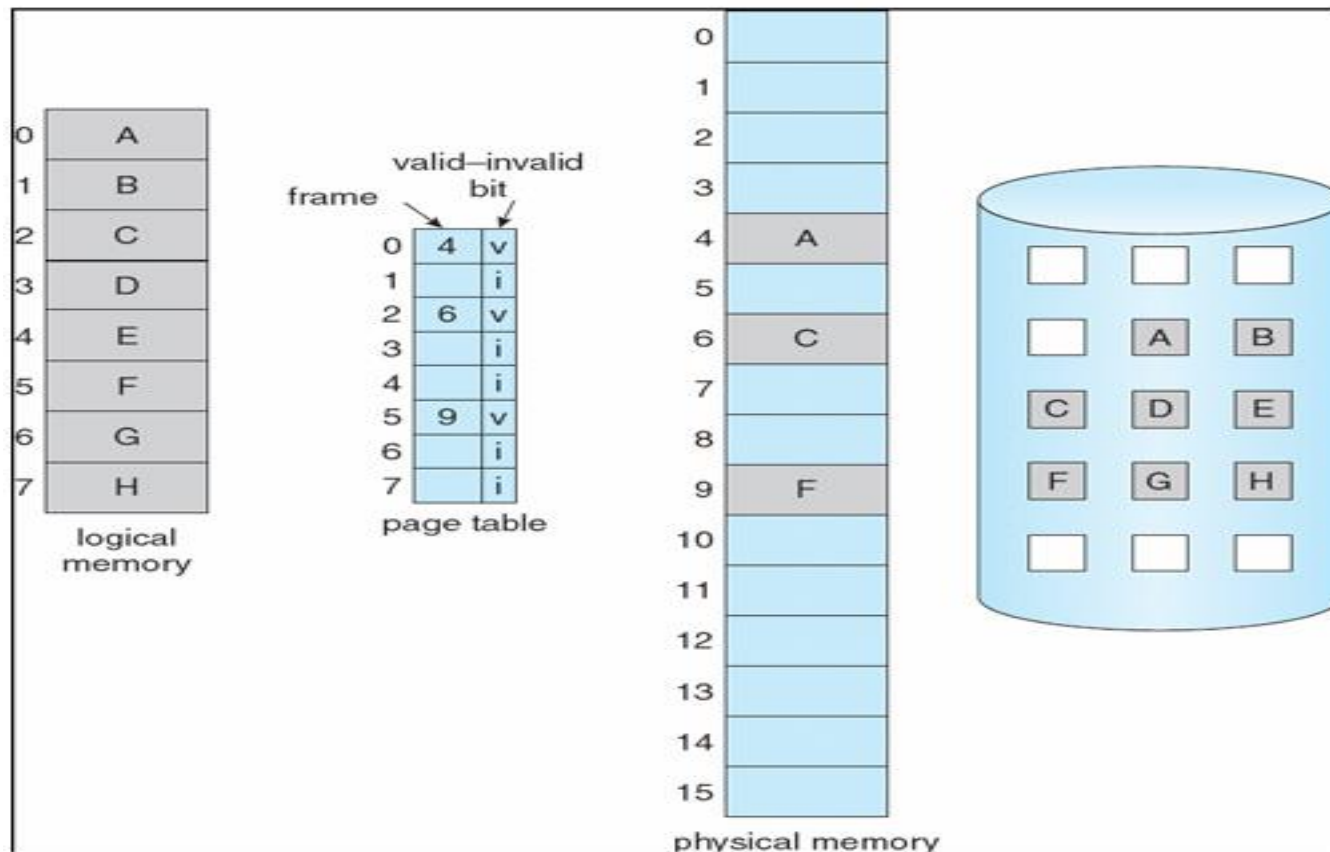
- Bring a page into memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
- With Demand-Paged virtual memory, pages are only loaded when they are demanded during program execution; pages that are never accessed are thus never loaded into physical memory.
- It is similar to paging system with swapping where processes reside in secondary memory.
- Lazy swapper concept is used in Demand Paging.
- **Lazy swapper**
 - Never swaps a page into memory unless that page will be needed
 - Swapper that deals with pages is a **pager**

Transfer of a Paged Memory to Contiguous Disk Space

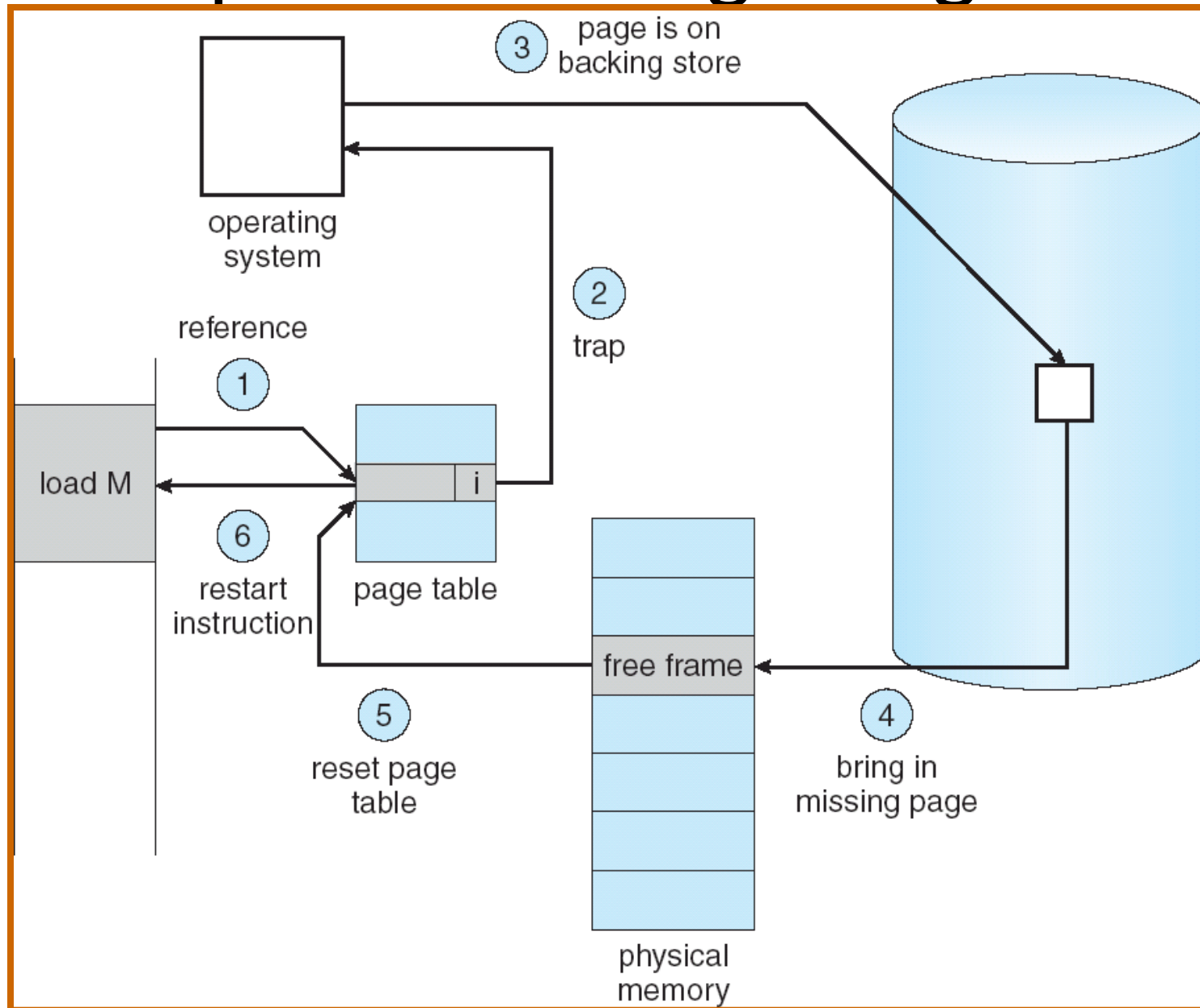


Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** $\Rightarrow$ in-memory, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:



Steps in Handling a Page Fault



Contd..

The Procedure for handling this page fault is :

1. We check an internal table .
2. If the reference was invalid, we terminate the process. If it was valid, but we have not yet brought in that page, we now page it in.
3. We find a free frame.
4. We schedule a disk operation to read the desired page into the newly allocated frame.
5. When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.

Contd..

- In the extreme case, we can start executing a process with *no* pages in memory.
- When the operating system sets the instruction pointer to the first instruction of the process, which is on a non-memory resident page, the process immediately faults for the page.
- After this page is brought into memory, the process continues to execute, faulting as necessary until every page that it needs is in memory.
- At that point, it can execute with no more faults. This scheme is **Pure Demand Paging**: Never bring a page into memory until it is required.

Contd..

- The hardware to support demand paging is the same as the hardware for paging and swapping:
 1. **Page table:** This table has the ability to mark an entry invalid through a valid-invalid bit or special value of protection bits.
 2. **Secondary memory:** This memory holds those pages that are not present in main memory. It is a high-speed disk and also known as swap device, and the section of disk used for this purpose is known as a swap space.

Performance of Demand Paging

❓ Page Fault Rate $0 \leq p \leq 1$

❓ if $p = 0$ no page faults

❓ if $p = 1$, every reference is a fault

❓ Effective Access Time (EAT)

$$\text{EAT} = (1 - p) \times \text{memory access}$$

$$+ p * (\text{page fault overhead}$$

$$+ \text{swap page out}$$

$$+ \text{swap page in}$$

$$+ \text{restart overhead})$$

} Page
fault
time

Performance Example

- *Memory access time* = 100 nanosec = 10^{-7}
- *Page fault overhead* = 25 millisec = 0.025
- *Page fault rate* = $1/1000 = 10^{-3}$
- $EAT = (1-p) * 10^{-7} + p * (0.025)$
 $= (0.999) * 10^{-7} + 10^{-3} * 0.025$
 $\cong 25 \text{ microseconds per reference!}$
- I.e.,
 $= 250 * \text{memory access time!}$

Demand Paging Example

❑ Memory access time = 200 nanoseconds

❑ Average page-fault service time = 8 milliseconds

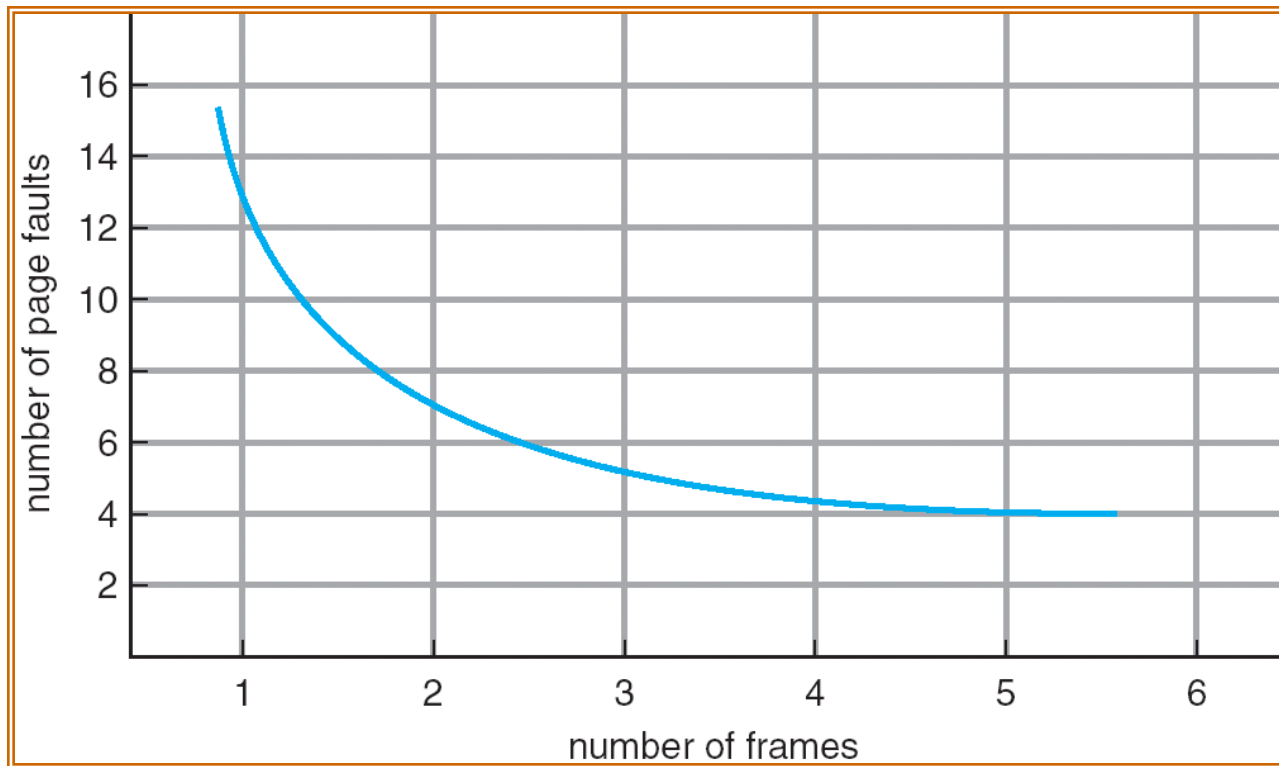
❑
$$\begin{aligned} \text{EAT} &= (1 - p) \times 200 + p (8 \text{ milliseconds}) \\ &= (1 - p) \times 200 + p \times 8,000,000 \\ &= 200 + p \times 7,999,800 \end{aligned}$$

❑ If one access out of 1,000 causes a page fault, then
EAT = 8.2 microseconds.

This is a slowdown by a factor of 40!!

Page Replacement Algorithms

- Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string

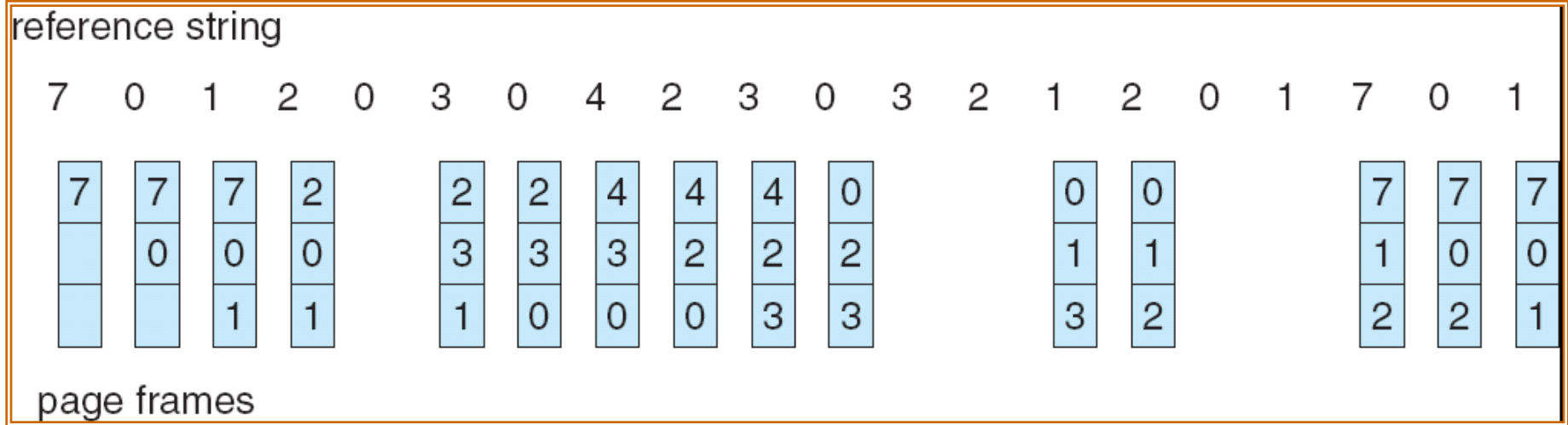


Page Replacement Algorithms

- First–In First–Out (**FIFO**)
- **Optimal**
- Least Recently Used (**LRU**)
- Second Chance (**Clock**)

FIFO Page Replacement Algorithm

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.



First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)

1	1	4	5	
2	2	1	3	9 page faults
3	3	2	4	

- 4 frames

1	1	5	4	
2	2	1	5	10 page faults
3	3	2		
4	4	3		

Belady's Anomaly: more frames $\Rightarrow$ more page faults

Optimal Algorithm

- Replace page that will not be used for longest period of time
- Example:

Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
4 frames

1
2
3
4

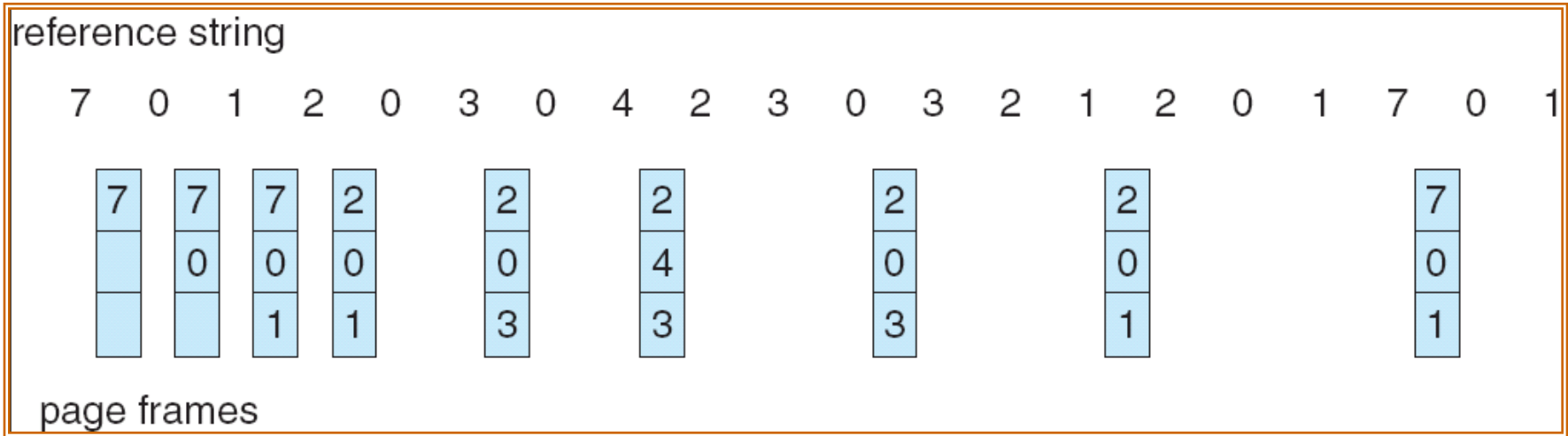
4

6 page faults

5

Optimal Page Replacement

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.



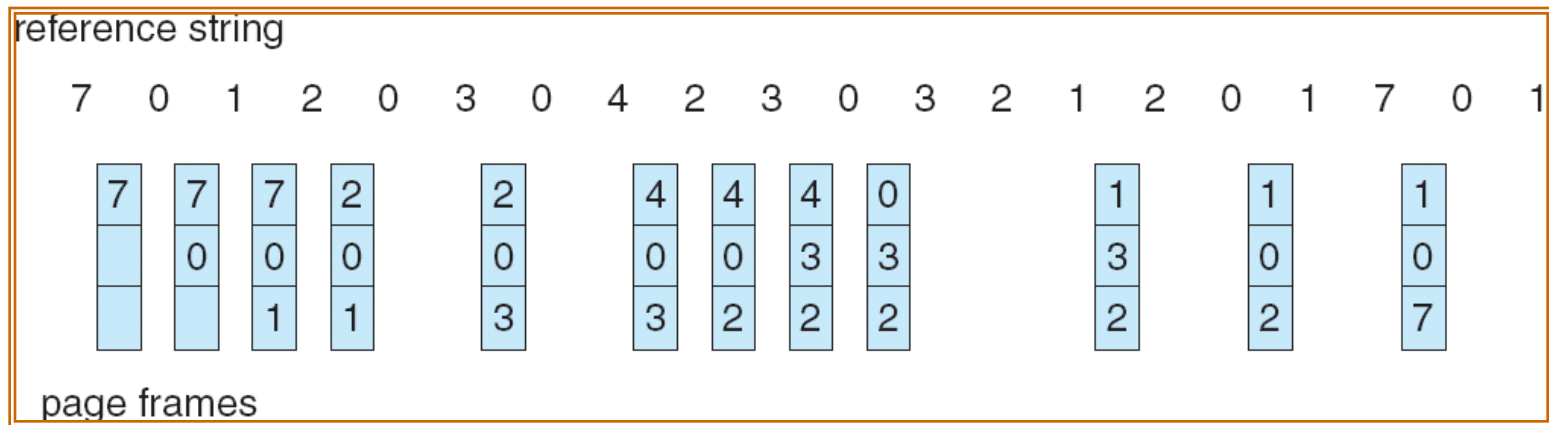
Least Recently Used (LRU) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, **5**, 1, 2, **3**, **4**, **5**
- Counter implementation
 - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be changed, look at the counters to determine which are to change

1	1	1	1	5
2	2	2	2	2
3	5	5	4	4
4	4	3	3	3

LRU Page Replacement

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.



- Stack implementation

Keep a stack of page numbers in a double link form:

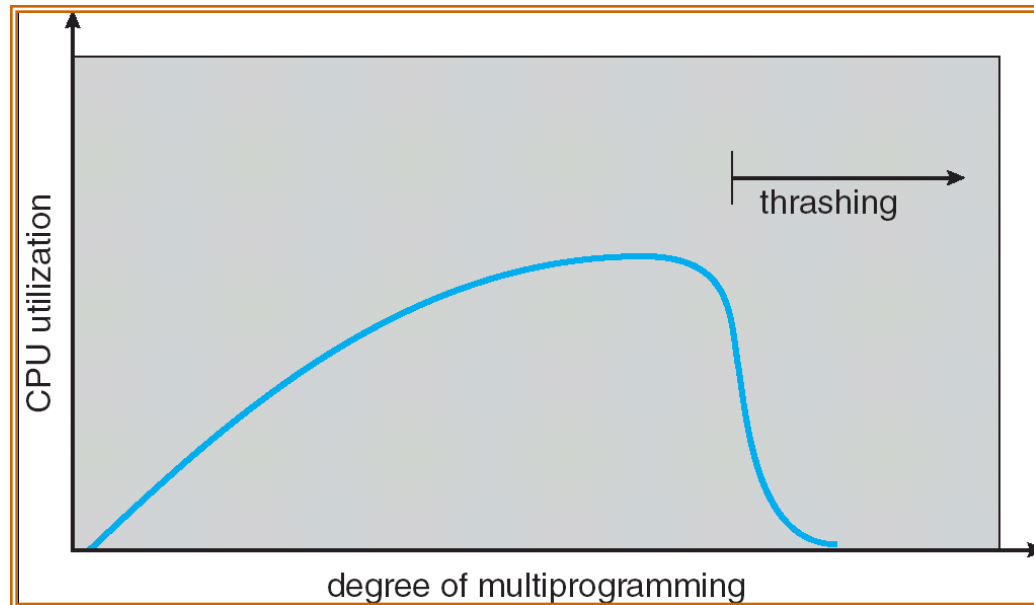
- Page referenced:

- Move it to the top
- requires 6 pointers to be changed

- No search for replacement

Thrashing

A process is busy swapping pages in and out



- We can limit the effect of thrashing by using a local replacement algorithm.
- To prevent Thrashing, we must provide a process as many frames as it needs.
- But how to calculate the required frame?
- The solution is by using working set theory.
- The working set theory starts by looking at how many frames a process is actually using. This approach defines the locality model of process execution

The following are the two methods to control thrashing.

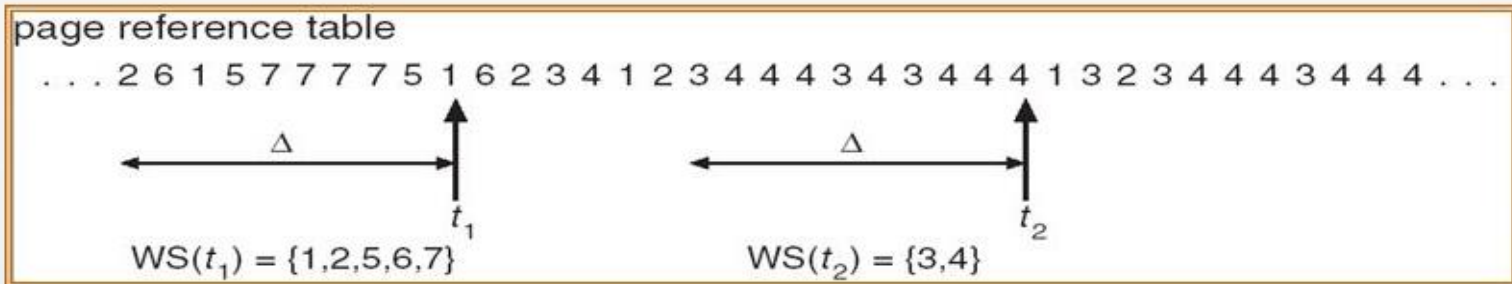
1. Working set model
2. Page-fault frequency

1. Working-set model.

- It is based on the assumption of locality.
- This model uses a parameter, Δ , to define the working-set window.
- The idea is to examine the most recent Δ page references is the working set.
- If a page is in active use, it will be in the working set. If it is no longer being used, it will drop from the working set Δ time units after its last reference.
- It is approximation of program's locality.

Working-Set Model

- $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
Example: 10,000 instruction



- WSS_i (working set size of Process P_i) = total number of pages referenced in the most recent Δ (varies in time)
 - if Δ too small will not encompass entire locality
 - if Δ too large will encompass several localities
 - if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \sum WSS_i \equiv$ total demand frames of all processes
- if $D > m$ (total number of available frames) $\Rightarrow$ Thrashing
- Policy: if $D > m$, then suspend one of the processes



2. Page-Fault Frequency

- The working set model is successful, and knowledge of the working set can be useful for prepaging, but it seems a clumsy way to control thrashing.
- A strategy that uses the page-fault frequency (PFF) takes a more direct approach.
- When the PF rate is too high, we know that process needs more frames. Conversely, if the page-fault rate is too low, then the process may have too many frames.

Contd..

- We can establish upper and lower bounds on the desired PF rate.
- If the actual page-fault rate exceeds the upper limit, we allocate the process another frame; if the page-fault rate falls below the lower limit, we remove a frame from the process. Thus, we can directly measure and control the page-fault rate to prevent thrashing.

Contd..

- As with the working-set strategy, we may have to suspend a process. If the page-fault rate increases and no free frames are available, we must select some process and suspend it.
- The freed frames are then distributed to processes with high page-fault rates.